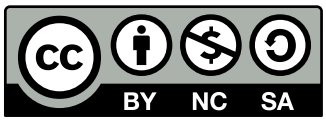
Aalto University
School of Science

Master's Programme in Computer, Communication and Information Sciences

Improving Retrieval-Augmented Generation with LLM-as-a-Judge Evaluation

Aleksi Hämäläinen

Master's Thesis
2025



Aalto University
School of Science

Author Aleksi Hämäläinen

Title Improving Retrieval-Augmented Generation with LLM-as-a-Judge Evaluation

Degree programme Computer, Communication and Information Sciences

Major Machine Learning, Data Science and Artificial Intelligence

Supervisor Prof. Alex Jung

Advisor Dr. Michael Samarin

Date 16 November 2025 **Number of pages** 88 **Language** English

Abstract

This thesis investigates how retrieval-augmented generation (RAG) pipelines can be systematically optimized and evaluated. RAG combines large language models (LLMs) with external retrieval systems, enabling them to access new information beyond their training data. This approach improves factual accuracy, reduces hallucinations, and allows LLMs to remain useful in fast-evolving domains. The study presents a step-by-step optimization of key retrieval components, including document parsing and chunking, embedding selection, hybrid search, query optimization, and reranking, and evaluates their effects using Ragas, an automated LLM-as-a-judge framework. Experiments were conducted on a corpus of machine learning research papers, representing a knowledge-intensive and rapidly changing field.

Iterative optimization led to substantial gains in retrieval quality, with context precision and recall increasing from 0.811 and 0.725 to 0.949 and 0.933, respectively. Among all components, the chunking strategy had the largest impact: larger chunks performed best, while overlap offered no clear benefit. Hybrid search consistently outperformed both semantic and keyword approaches. Among reranking techniques, only the LLM-based listwise method improved retrieval performance, whereas query optimization did not lead to measurable gains.

The study also compares open-source and proprietary embedding models and LLMs and examines how well leaderboard rankings predict real-world RAG performance. OpenAI's proprietary models achieved the highest Ragas scores, while open-source alternatives, including `gte-large-en-v1.5` for embeddings and Llama 3.1 8B for generation, achieved competitive results. Evaluation results further showed that metrics varied between LLM evaluators and that measures such as answer semantic similarity were weak indicators of factual correctness.

Overall, the findings show that effective retrieval design and rigorous evaluation can narrow the performance gap between open-source and proprietary RAG systems. The results highlight the importance of systematic, domain-specific optimization and emphasize the need for more reliable automated evaluation methods for RAG.

Keywords retrieval-augmented generation, large language models, llm-as-a-judge, question answering, natural language processing, information retrieval

Aalto-yliopisto
Perustieteiden
korkeakoulu

Tekijä Aleksi Hämäläinen

Työn nimi Hakupohjaisen generoinnin parantaminen kielimallipohjaisen arvioinnin avulla

Koulutusohjelma Computer, Communication and Information Sciences

Pääaine Machine Learning, Data Science and Artificial Intelligence

Työn valvoja Prof. Alex Jung

Työn ohjaaja FT Michael Samarin

Päivämäärä 16.11.2025 **Sivumäärä** 88 **Kieli** englanti

Tiivistelmä

Tämä diplomityö tarkastelee, miten hakupohjaisen generoinnin (retrieval-augmented generation, RAG) prosesseja voidaan systemaattisesti optimoida ja arvioida. RAG yhdistää suuret kielimallit ulkoisiin hakujärjestelmiin, jolloin mallit voivat hyödyntää uutta tietoa opetusdatansa ulkopuolelta. Tämä parantaa vastausten tarkkuutta, vähentää hallusinaatioita ja mahdollistaa kielimallien tehokkaan käytön nopeasti kehittyvillä aloilla. Tutkimuksessa esitetään vaiheittainen optimointimenetelmä keskeisille hakukomponenteille: dokumenttien jäsentämiselle ja paloittelulle, upotusmallin valinnalle, hybridihakumenetelmälle, kyselyiden optimoinnille sekä haettujen palasten uudelleensijoittelulle. Näiden vaikutuksia arvioitiin Ragas-kirjastolla, joka hyödyntää automatisoitua kielimallipohjaista arviointia. Kokeet suoritettiin koneoppimisen tutkimusartikkeleista koostuvalla aineistolla, joka edustaa tietointensiivistä ja nopeasti uudistuvaa alaa.

Iteratiivinen optimointi paransi merkittävästi haun laatua: kontekstin tarkkuus ja saanti kasvoivat arvoista 0,811 ja 0,725 arvoihin 0,949 ja 0,933. Paloittelustrategialla oli suurin vaikutus: suuremmat palat tuottivat parhaat tulokset, kun taas päällekkäisyys ei parantanut suorituskykyä. Hybridihaku päihitti sekä semanttiset että avainsanapohjaiset menetelmät. Uudelleensijoittelutekniikoista vain kielimallipohjainen menetelmä paransi tuloksia, kun taas kyselyiden optimointi ei tuonut hyötyä.

Tutkimuksessa verrattiin myös avoimen ja suljetun lähdekoodin upotus- ja kielimalleja sekä tarkasteltiin, kuinka hyvin tulostaulukot ennustavat RAG-järjestelmien todellista suorituskykyä. OpenAI:n suljetut mallit saavuttivat korkeimmat Ragas-pisteet, mutta avoimen lähdekoodin vaihtoehdot, kuten `gte-large-en-v1.5` ja Llama 3.1 8B, tuottivat kilpailukykyisiä tuloksia. Arviointitulokset osoittivat lisäksi, että Ragas-pisteet vaihtelivat eri kielimalliarvioijien välillä ja että tietyt metriikat, kuten vastauksen semanttinen samankaltaisuus, korreloivat heikosti faktuaalisen oikeellisuuden kanssa.

Tulokset osoittavat, että tehokas optimointi ja huolellinen arviointi voivat kaventaa avoimen ja suljetun lähdekoodin RAG-järjestelmien suorituskykyeroa. Tutkimus korostaa sovellusaluekohtaisen optimoinnin merkitystä sekä tarvetta kehittää luotettavampia, automatisoituja arviointimenetelmiä RAG-järjestelmille.

Avainsanat hakupohjainen generointi, suuret kielimallit, kielimallipohjainen arviointi, kysymyksiin vastaaminen, luonnollisen kielen käsittely, tiedonhaku

Preface

I would like to thank my supervisor, Alex Jung, and advisor, Michael Samarin, for their guidance. I also want to thank my parents and sister for supporting me throughout the thesis process and my studies.

Helsinki, 16 November 2025

Aleksi Hämäläinen

Contents

Abstract 3

Abstract (in Finnish) 4

Preface 5

Contents 6

Abbreviations 8

1 Introduction 9

2 Background 11
2.1 Large Language Models . . . 11
2.1.1 Tokenization . . . 12
2.1.2 Transformer Architecture . . . 12
2.1.3 Encoder-Only Models . . . 15
2.1.4 Decoder-Only Models . . . 16
2.1.5 Scaling Laws and Emergent Abilities . . . 17
2.1.6 Prompt Engineering . . . 17
2.1.7 Limitations . . . 18
2.2 Information Retrieval . . . 20
2.2.1 Sparse Retrieval . . . 20
2.2.2 Dense Retrieval . . . 22
2.3 Nearest Neighbor Search . . . 24
2.3.1 Hierarchical Navigable Small World . . . 24
2.4 Retrieval-Augmented Generation . . . 25
2.4.1 Original RAG Framework . . . 26
2.4.2 Modern RAG Paradigms . . . 27
2.5 Evaluation of Retrieval-Augmented Generation . . . 30
2.5.1 Evaluation Challenges . . . 30
2.5.2 Evaluation Aspects . . . 30
2.5.3 Evaluation Metrics . . . 31
2.5.4 LLM-as-a-Judge . . . 31
2.5.5 Ragas . . . 32

3 Methods 33
3.1 Experimental Setup . . . 33
3.2 Data Collection . . . 34
3.3 Baseline RAG Pipeline . . . 35
3.4 RAG Pipeline Components . . . 36
3.4.1 Document Parsing and Chunking . . . 36
3.4.2 Embedding Models . . . 39
3.4.3 Hybrid Search . . . 41

3.4.4 Query Optimization . . . 41
3.4.5 Reranking . . . 42
3.4.6 Large Language Models . . . 43
3.5 Evaluation . . . 46
3.5.1 Evaluation Metrics . . . 46
3.5.2 QA Test Set Generation . . . 48
3.5.3 Evaluation Strategy . . . 50

4 Results **51**
4.1 Baseline Performance . . . 51
4.2 Retrieval Optimization . . . 52
4.2.1 Chunking Strategies . . . 52
4.2.2 Embedding Models . . . 53
4.2.3 Hybrid Search . . . 54
4.2.4 Query Optimization . . . 55
4.2.5 Reranking . . . 55
4.2.6 Summary of Retrieval Improvements . . . 55
4.3 Large Language Models . . . 56
4.4 Optimized RAG Pipeline . . . 57
4.5 Comparison of Baseline and Optimized RAG Pipelines . . . 58
4.6 Evaluation Robustness . . . 60

5 Discussion **66**
5.1 Optimizing Retrieval in RAG Pipelines . . . 66
5.2 Comparing Open-Source and Proprietary Models in RAG . . . 68
5.3 Evaluating RAG with LLM-as-a-Judge . . . 70
5.4 Limitations . . . 71
5.5 Future Work . . . 72

6 Conclusion **74**

References **75**

Abbreviations

AI	Artificial Intelligence
ANNS	Approximate Nearest Neighbor Search
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BGE	BAAI General Embedding
BM25	Best Matching 25
BPE	Byte Pair Encoding
ColBERT	Contextualized Late Interaction over BERT
CoT	Chain-of-Thought
DPR	Dense Passage Retriever
GPT	Generative Pre-Trained Transformer
GTE	General Text Embedding
HNSW	Hierarchical Navigable Small World
HyDE	Hypothetical Document Embeddings
LLM	Large Language Model
LSTM	Long Short-Term Memory
ML	Machine Learning
MLM	Masked Language Modeling
MTEB	Massive Text Embedding Benchmark
NLM	Neural Language Model
NLP	Natural Language Processing
NSP	Next Sentence Prediction
PLM	Pre-Trained Language Model
PPO	Proximal Policy Optimization
QA	Question Answering
RAG	Retrieval-Augmented Generation
Ragas	Retrieval-Augmented Generation Assessment
RLHF	Reinforcement Learning from Human Feedback
RNN	Recurrent Neural Network
RoBERTa	Robustly Optimized BERT Pretraining Approach
SBERT	Sentence-BERT
SFT	Supervised Fine-Tuning
SLM	Statistical Language Model
TF-IDF	Term Frequency-Inverse Document Frequency
VLM	Vision Language Model

1 Introduction

Recent advances in large language models (LLMs) have transformed how knowledge is accessed, created, and applied across disciplines. Since the release of ChatGPT [1] in November 2022, interest in LLMs has grown rapidly, and they have been adopted across domains such as healthcare, education, law, finance, and scientific research [2]. Despite these transformative capabilities, LLMs face fundamental limitations. Their training data is frozen at a fixed point in time, leaving them unaware of recent developments. They also struggle with long-tail knowledge, referring to rare facts that appear infrequently in training data [3]. As a result, LLMs often perform poorly on tasks that require accurate, up-to-date information, sometimes producing hallucinations: outputs that are fluent yet factually incorrect [4].

One promising approach to addressing these limitations is retrieval-augmented generation (RAG) [5]. By combining LLMs with external retrieval systems, RAG enables models to ground their responses in relevant, up-to-date documents. This approach is particularly valuable in fast-moving domains such as scientific research, where new knowledge emerges continuously and static models quickly become outdated.

Building effective RAG systems, however, is far from straightforward. Even small design choices in document parsing, chunking, embedding, or ranking can significantly affect performance [6]. Beyond retrieval design, model selection plays a critical role in both retrieval and generation. Proprietary LLMs accessed through APIs typically achieve strong results, but they are expensive to use at scale, raise privacy concerns, and may be unsuitable in regulated environments. Open-source models, by contrast, are more transparent and affordable, yet their effectiveness within RAG pipelines remains uncertain. Understanding whether smaller open-source models can achieve competitive performance is essential for making RAG more practical and cost-effective. Although several public leaderboards evaluate both embedding models [7] and LLMs [8, 9], it remains unclear how well such benchmark results reflect real-world RAG performance.

Finally, evaluating RAG systems presents distinct challenges. Human evaluation remains the gold standard for assessing generation quality, but it is slow and costly. Common metrics such as BLEU and ROUGE, which rely on lexical overlap, also correlate poorly with human judgments [10]. Moreover, many organizations lack domain-specific question answering (QA) datasets for evaluating RAG systems, making systematic benchmarking difficult. This motivates the use of automated evaluation frameworks such as LLM-as-a-judge [11], where LLMs assess the quality of outputs produced by other LLMs. Ragas [12] is one such framework, providing an automated, LLM-based approach for both generating test data and evaluating RAG system performance.

To address these challenges, this thesis presents a systematic, step-by-step evaluation of a RAG pipeline. We begin with a simple baseline and progressively optimize its components, measuring their effects using Ragas. Experiments are conducted on a corpus of machine learning research papers, chosen as a representative example of a knowledge-intensive, domain-specific dataset. Rather than exhaustively testing all

possible methods, the goal is to identify which components yield the largest gains and to assess whether optimized retrieval enables smaller open-source models to compete with proprietary ones.

We focus on three research questions:

1. How can retrieval performance in RAG be systematically improved through component-level optimization?
2. How do open-source embedding models and LLMs compare with proprietary ones, and how well do leaderboard scores reflect their real-world performance?
3. How can LLM-as-a-judge frameworks, such as Ragas, be applied to evaluate RAG systems?

In summary, the contributions of this thesis are threefold. First, it investigates how retrieval in RAG pipelines can be systematically optimized, analyzing the effects of document parsing and chunking, embeddings, hybrid search, query optimization, and reranking. Second, it compares open-source and proprietary models, examining whether leaderboard performance predicts real-world effectiveness in RAG. Third, it evaluates RAG pipelines with LLM-as-a-judge, using Ragas for both automated test set generation and system evaluation, and examining its reliability, strengths, and limitations.

The rest of this thesis is organized as follows. Section 2 provides the theoretical background for RAG, covering core concepts of large language models, retrieval techniques, and evaluation methods. Section 3 describes the methodology, followed by Section 4, which presents the experimental findings. Section 5 interprets the results, discusses their implications, analyzes the limitations of this work, and outlines directions for future research. Finally, Section 6 summarizes the key findings of the thesis.

2 Background

This section provides the theoretical foundation for understanding the key components of retrieval-augmented generation (RAG) and the approaches used to evaluate it. Section 2.1 introduces large language models (LLMs), describing their architecture, scaling behavior, and limitations. Section 2.2 gives an overview of information retrieval and text representations for retrieval, and Section 2.3 explains how efficient retrieval is enabled through approximate nearest neighbor search (ANNS). Section 2.4 outlines the original RAG framework and its modern variants, while Section 2.5 reviews methods for evaluating RAG systems and the challenges involved.

2.1 Large Language Models

Language modeling aims to predict the next token given the previous ones. Through this process, models learn underlying linguistic patterns and structures, which in turn allows them to generate coherent text and perform a wide range of language understanding tasks. Over time, language models have evolved through several generations, each expanding their capacity to solve increasingly complex tasks [2]. Early statistical language models (SLMs) relied on n-gram models that predicted each word based on a limited number of preceding ones. Neural language models (NLMs) replaced these hand-crafted statistics with models that learn directly from data using architectures such as recurrent neural networks (RNNs).

The idea of pre-training language models on large text corpora before fine-tuning them for downstream tasks led to the emergence of pre-trained language models (PLMs). The introduction of the transformer architecture enabled this paradigm to scale further, leading to the development of transformer-based PLMs, such as BERT [13] and the early GPT models [14, 15]. These models learned contextual representations that could be fine-tuned for a wide range of NLP tasks, making them transferable task solvers [2]. Scaling up transformer-based PLMs in terms of parameters, data, and compute has led to a new generation known as large language models (LLMs). Trained on vast datasets, they can generate coherent and contextually appropriate language while solving a wide range of tasks without task-specific fine-tuning. LLMs have become general-purpose task solvers capable of performing diverse real-world tasks through prompting [2]. A prominent example is ChatGPT [1], which adapts LLMs for dialogue and demonstrates the strong language understanding and generation abilities of modern large-scale models.

This section introduces the key components of large language models relevant to this work. We begin with tokenization and the transformer architecture before describing encoder-only and decoder-only model types. We then discuss model scaling, prompting, and the limitations of LLMs that motivate the use of retrieval-augmented generation.

2.1.1 Tokenization

Tokenization is a preprocessing step performed before feeding text into a language model, in which raw text is divided into smaller units known as tokens [2]. A traditional and straightforward approach is word-based tokenization, where each word is treated as a token. However, this method struggles with out-of-vocabulary words, referring to terms that do not appear in the training data, and often results in a large vocabulary containing many low-frequency terms. It is also unsuitable for languages such as Chinese, where words are not separated by spaces.

Nowadays, a commonly used method for tokenization is subword tokenization [2]. One of the most widely adopted subword tokenizers is Byte Pair Encoding (BPE), which was first proposed as a data compression technique [16] and later adapted for tokenization by Sennrich et al. [17]. BPE has been adopted in many LLMs, such as the GPT series [14, 15, 18].

The BPE algorithm begins with a vocabulary of individual characters and iteratively merges the most frequent consecutive token pairs in the corpus until a predefined vocabulary size is reached. The final vocabulary size equals the number of initial symbols plus the number of merge operations, which serves as a hyperparameter of the algorithm.

A closely related method is WordPiece [19], introduced by Google and later used in BERT [13], which follows a similar principle but selects merges that maximize the likelihood of the training data. Once the vocabulary is constructed, the resulting tokens serve as the input to transformer-based architectures.

2.1.2 Transformer Architecture

The transformer architecture [20], introduced by Vaswani et al. in 2017, revolutionized natural language processing (NLP) by enabling highly parallel and scalable sequence modeling. Earlier sequence models based on RNNs, such as long short-term memory (LSTM) networks [21], processed text token by token, maintaining a hidden state that evolved over time. While effective for short sequences, their sequential nature limited parallelization and made it difficult to capture long-range dependencies.

The introduction of attention mechanisms [22] addressed these limitations by allowing models to focus on relevant parts of the input sequence. The transformer generalizes this idea through a fully self-attentive architecture that eliminates recurrence entirely, enabling all tokens to attend to one another in parallel. This shift marked a major step forward in both efficiency and representational power.

The transformer uses an encoder-decoder architecture, as shown in Figure 1. The encoder maps an input sequence of tokens $(x_1, \ldots, x_n)$ into a sequence of contextual representations $\mathbf{z} = (z_1, \ldots, z_n)$ while the decoder autoregressively generates an output sequence $(y_1, \ldots, y_m)$, conditioning each prediction on both $\mathbf{z}$ and previously generated tokens.

Before entering the encoder or decoder, each token is first converted into a dense vector of dimension d_{model} using a learned embedding layer. Because the transformer processes all tokens in parallel rather than sequentially, it has no inherent notion of

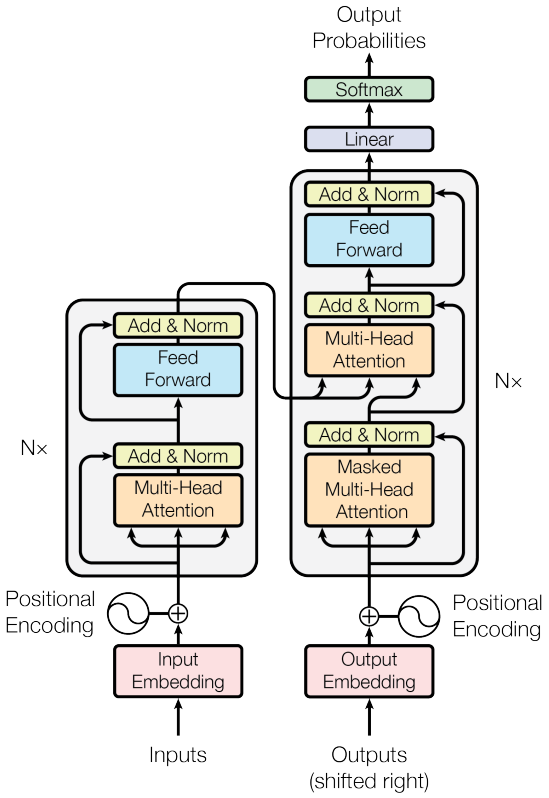


Figure 1: The transformer encoder-decoder architecture [20]. The encoder processes the input sequence into contextual representations, while the decoder autoregressively generates the output sequence using both the encoded context and previously generated tokens.

order. To address this, positional encodings are added to the embeddings to provide information about token positions within the sequence.

Vaswani et al. [20] proposed sinusoidal positional encodings that allow the model to generalize to sequences longer than those seen during training:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right)$$
$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right),$$

where pos is the position of a token in the input sequence and i is the dimension index within the embedding vector.

Each encoder layer consists of a multi-head self-attention mechanism, which enables the model to relate tokens to each other regardless of their distance, followed by a position-wise feed-forward network that transforms each representation independently. Both sub-layers employ residual connections and layer normalization, ensuring stable and efficient training. The decoder extends this structure with an additional cross-attention sub-layer that attends to the encoder outputs, allowing it to incorporate

contextual information from the input sequence. To preserve the autoregressive property required for text generation, a causal mask is applied to the decoder's self-attention to prevent tokens from attending to future positions.

The core operation of the transformer is the self-attention mechanism, which allows the model to dynamically weight the importance of different tokens when constructing contextual representations. Each token embedding is linearly projected into query, key, and value representations, forming the matrices Q, K, and V. The attention mechanism computes a weighted sum of the values based on the similarity between queries and keys:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V,$$

where d_k is the dimension of the key vectors. This operation enables each token to incorporate information from all others in the sequence, capturing dependencies regardless of their relative positions.

To improve expressiveness, the transformer uses multi-head attention, where multiple attention operations are performed in parallel using different learned projections. Each head attends to information from a different representation subspace, and the results are concatenated and linearly transformed:

$$\text{MultiHead}(Q, K, V) = \text{Concat}(\text{head}_1, ..., \text{head}_h)W^O,$$

with each attention head computed as:

$$\text{head}_i = \text{Attention}(QW_i^Q, KW_i^K, VW_i^V),$$

where W_i^Q, W_i^K, W_i^V and W^O are the parameter matrices of the model. This design enables the model to jointly capture diverse linguistic relationships, such as syntactic and semantic dependencies. The scaled dot-product attention and multi-head attention mechanisms are illustrated in Figure 2.

Finally, the decoder output is passed through a linear layer followed by a softmax function to produce a probability distribution over the vocabulary. A simple method for sampling tokens from this distribution is greedy search, which selects the token with the highest predicted probability, considering the tokens produced so far. Another widely used approach is temperature sampling, which we use in this thesis. In this method, the temperature parameter controls the level of randomness during generation: reducing the temperature makes the model more likely to choose high-probability tokens and less likely to sample low-probability ones [2]. When the temperature is set to 1, the model performs random sampling, whereas as the temperature approaches 0, the behavior converges toward greedy search.

The encoder-decoder design of the original transformer architecture served as the foundation for later models, such as BART [23], which was used in the original RAG framework discussed in Section 2.4.1. Building on this foundation, subsequent models have specialized toward either encoder-only or decoder-only architectures, which are introduced in the following sections.

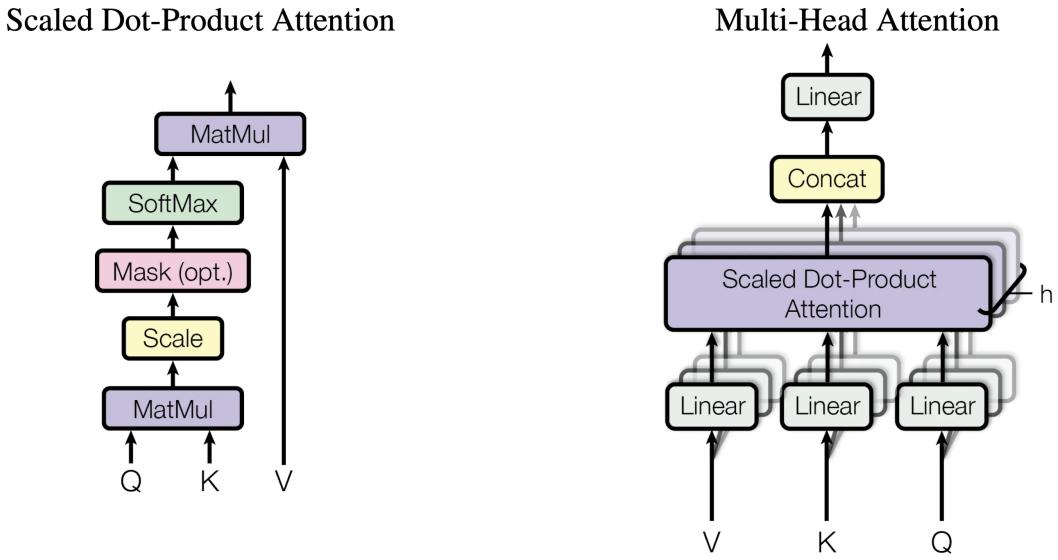


Figure 2: Scaled dot-product attention and multi-head attention mechanisms [20]. The left diagram illustrates how attention weights are computed using queries, keys, and values, while the right shows how multiple attention heads operate in parallel and are combined through concatenation and a linear projection.

2.1.3 Encoder-Only Models

One of the most influential encoder-only transformer models is BERT (Bidirectional Encoder Representations from Transformers) [13], released by Google in 2018. The original BERT architecture comes in two sizes: $\text{BERT}_{\text{BASE}}$, with 110 million parameters, and $\text{BERT}_{\text{LARGE}}$, with 340 million parameters.

A key innovation of BERT is its bidirectional context representation. Unlike earlier language models that processed text in a unidirectional manner, BERT leverages a masked language modeling (MLM) objective during pre-training. In this unsupervised task, a portion of the input tokens is randomly masked, and the model is trained to predict the masked tokens based on the surrounding context. This allows BERT to learn deep, context-dependent language representations from both left and right contexts simultaneously.

In addition to MLM, BERT is pre-trained with a next sentence prediction (NSP) objective. In this task, the model receives pairs of sentences and learns to predict whether the second sentence logically follows the first in the original text. This additional task helps BERT capture inter-sentence relationships, improving performance on downstream tasks such as question answering and natural language inference.

BERT employs several special tokens in its input format. Each input sequence begins with a `[CLS]` token, whose final hidden representation is typically used for classification tasks. For sentence-pair inputs, a `[SEP]` token marks the boundary between sentences. The final input embeddings are obtained by summing three components: token embeddings, which represent individual tokens, segment embeddings, which indicate whether a token belongs to the first or second segment, and positional embeddings,

which encode the position of each token in the sequence. This composite representation is then fed into the transformer encoder.

After pre-training, BERT can be fine-tuned for a wide range of downstream NLP tasks by adding a small, task-specific output layer and continuing training on labeled data. This flexibility and efficiency in transfer learning have contributed significantly to BERT's widespread adoption in natural language processing.

Several improved variants of BERT have since been developed. One notable example is RoBERTa (Robustly Optimized BERT Approach) [24]. RoBERTa retains the same underlying architecture as BERT but enhances performance through several modifications to the pre-training procedure. These include training on larger datasets with larger batch sizes and longer training durations, using longer input sequences, dynamically varying the masking pattern applied to the training data, and removing the NSP objective. As a result, RoBERTa achieves stronger performance across a wide range of natural language understanding benchmarks and has become a widely adopted foundation for modern encoder-only transformer models.

2.1.4 Decoder-Only Models

Another important category of transformer architectures is decoder-only models. In the literature, the term large language model (LLM) commonly refers to models of this type [2]. In this thesis, the term LLM will likewise be used to denote decoder-only transformer models. A representative example of this architecture is the Generative Pre-trained Transformer (GPT) series developed by OpenAI [14, 15, 18, 25]. Other well-known open-source implementations include the Llama models developed by Meta [26, 27, 28] and the Gemma models introduced by Google [29, 30].

These models are pre-trained in an autoregressive manner on large-scale datasets using the language modeling objective: predicting the next token given the preceding tokens in a sequence [2]. The number of preceding tokens the model can attend to is limited by its context window, which defines the maximum number of tokens the model can process at once. This training setup enables them to learn rich language representations and general-purpose knowledge. Such models are often referred to as base models or foundation models.

While base models demonstrate strong generative capabilities, they are not inherently aligned with human preferences. To address this limitation, a common approach is to further fine-tune these models using human feedback [31]. One widely adopted method is reinforcement learning from human feedback (RLHF) [32], a multi-phase training procedure designed to improve model alignment. The process begins with supervised fine-tuning (SFT), in which the model is trained on a dataset containing prompts and high-quality human-generated responses. Following this, a reward model is constructed by collecting human preference data over multiple model outputs, allowing it to learn a proxy for human judgment. In the final phase, the model from the SFT step is optimized using a reinforcement learning algorithm such as proximal policy optimization (PPO) [33], where the reward model guides the learning process towards producing more preferred outputs. Models that undergo this fine-tuning process are commonly referred to as instruction-tuned models, as they are

specifically optimized to follow user instructions more effectively.

The performance of decoder-only models has improved rapidly, and the models have demonstrated remarkable performance across a wide range of tasks [2]. One of the key factors behind this success is the scaling of the model size. While the core architecture and pre-training objective of the GPT models have remained largely consistent, their size has increased significantly. GPT-1, introduced in 2018 [14], contained 117 million parameters. This was followed by GPT-2 in 2019 [15] with 1.5 billion parameters, and GPT-3 in 2020 [18], which scaled up to 175 billion parameters. The parameter count and training details of GPT-4, released in 2023 [25], have not been disclosed due to growing concerns around the safety of large language models and the increasingly competitive nature of the field. The following section examines how model scaling can be quantitatively modeled and how increasing scale has led to the emergence of new capabilities.

2.1.5 Scaling Laws and Emergent Abilities

Scaling laws [34, 35] are empirical relationships demonstrating that the performance of LLMs depends predictably on three main factors: model size, dataset size, and computational budget. These relationships enable researchers to estimate performance improvements when scaling up models, datasets, or compute. This predictability is especially valuable at large scales, where exhaustive model-specific tuning is impractical.

For example, GPT-4 [25] was developed using infrastructure and optimization approaches that behaved consistently across model sizes, allowing its performance to be anticipated from smaller, less computationally expensive models. While scaling laws are generally framed around language modeling loss, GPT-4 also showed that certain downstream capabilities, such as code generation, can be forecasted using these laws. However, not all tasks improve monotonically with scale. Some exhibit inverse scaling, where performance degrades as models grow larger, as demonstrated in the Inverse Scaling Prize [36].

In contrast, emergent abilities [37] refer to qualitative behaviors that arise abruptly once a model surpasses a certain scale threshold. These abilities cannot be predicted by extrapolating the performance of smaller models. Examples include in-context learning, instruction following, and step-by-step reasoning [2]. Such abilities are scale-dependent and often triggered by specific input conditions, such as carefully designed prompts. It has also been argued that some of these apparent emergent abilities arise from the choice of evaluation metric rather than from fundamental changes in model behavior, with nonlinear or discontinuous metrics producing the illusion of abrupt emergence [38]. In the next section, we explore how such abilities can be surfaced through prompting.

2.1.6 Prompt Engineering

Prompt engineering refers to the design and refinement of natural language prompts that effectively guide LLMs toward desired outputs [2]. A well-crafted prompt typically

includes a clear task description specifying the objective, input data representing the problem or question, and optional contextual information to support reasoning. Beyond structure, the phrasing, style, and tone of a prompt strongly influence model behavior and output quality.

Several principles have been proposed to enhance prompt effectiveness [2]. Task descriptions should be explicit and unambiguous to minimize irrelevant or incorrect responses. For complex problems, decomposing the task into smaller subtasks can guide the model toward stepwise solutions. Including a few annotated examples, an approach known as in-context learning, further improves model performance on intricate tasks.

In-context learning [18] allows LLMs to learn from examples presented directly in the prompt, without updating model weights. In contrast to training or fine-tuning a model, what has been learned during in-context learning is temporary. This capability is an emergent property: while GPT-3 [18] demonstrated strong in-context learning, smaller predecessors such as GPT-1 [14] and GPT-2 [15] did not. When a few examples are provided, the approach is called few-shot learning. One-shot learning involves a single example, while zero-shot learning provides none.

Instruction tuning has further advanced zero-shot performance by fine-tuning models on diverse task instructions [39]. This process teaches models to follow novel instructions without explicit examples, enabling improved generalization. The ability to generalize from instructions alone is also regarded as an emergent property, as it is not observed in smaller models.

Chain-of-Thought (CoT) prompting [40] extends few-shot prompting by including intermediate reasoning steps in each example. By encouraging explicit step-by-step reasoning, CoT prompting elicits another emergent ability and substantially improves logical reasoning performance compared to standard few-shot prompting. Remarkably, even without examples, appending the phrase "Let's think step by step" before each answer, an approach known as zero-shot-CoT [41], can trigger similar reasoning behaviors. Figure 3 illustrates the differences between few-shot prompting, few-shot CoT prompting, zero-shot prompting, and zero-shot-CoT prompting.

While advanced prompting methods substantially improve reasoning performance, they do not fully resolve the broader limitations of LLMs, particularly when reliability and factual accuracy are required.

2.1.7 Limitations

Two major challenges in applying LLMs to knowledge-intensive tasks are hallucination and knowledge recency [2]. Before the emergence of LLMs, hallucination typically referred to text generation that was illogical or not faithful to the original input. It was commonly divided into intrinsic and extrinsic hallucination [42]. Intrinsic hallucination occurs when the generated output contradicts the source content, whereas extrinsic hallucination arises when it includes information not verifiable against the source.

With the rise of LLMs, this taxonomy has evolved to capture new model behaviors and capabilities. Hallucinations are now classified as input-conflicting, context-

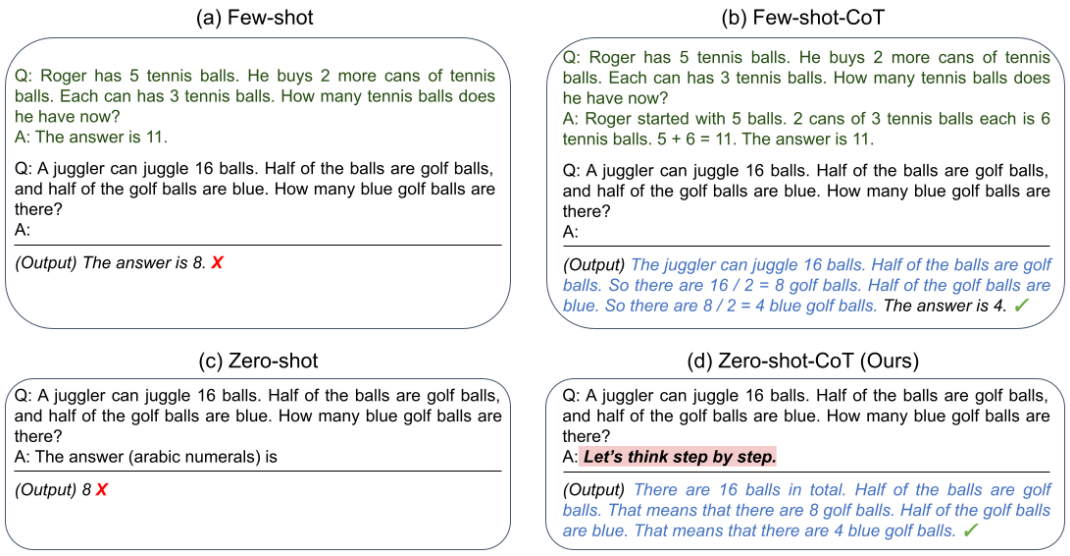


Figure 3: Comparison of prompting strategies for reasoning tasks [41]. Few-shot prompting provides examples without reasoning steps, while few-shot-CoT prompting adds intermediate reasoning to guide step-by-step problem solving. Zero-shot prompting provides no examples and often fails on reasoning tasks, whereas zero-shot-CoT prompting elicits reasoning behavior by appending "Let's think step by step", enabling more accurate multi-step answers.

conflicting, and fact-conflicting [4]. Input-conflicting hallucinations occur when generated content diverges from the user's prompt, context-conflicting hallucinations arise when the output contradicts previous model-generated text, and fact-conflicting hallucinations refer to content inconsistent with established world knowledge.

Recent studies primarily focus on fact-conflicting hallucinations, which pose the most persistent and challenging issues in current LLMs [4]. Unlike earlier task-specific models trained on limited data, LLMs are pre-trained on trillions of tokens and are expected to generalize across tasks, languages, and domains. Their outputs are often linguistically fluent and coherent, making false information appear highly convincing, sometimes even to expert evaluators.

Hallucinations can emerge at multiple stages of the LLM lifecycle [4]. During pre-training, models may absorb incorrect or incomplete information from noisy data, while overconfidence at inference can cause them to present false statements with unwarranted certainty. The alignment process, intended to fine-tune models to human preferences, may further encourage hallucinations when models attempt to satisfy user intent. Moreover, the sequential nature of token generation can amplify early mistakes, leading to hallucination snowballing.

Beyond hallucination, a closely related limitation is knowledge recency: LLMs cannot access information beyond their training cutoff [2]. A simple way to mitigate this is to periodically update the model with new data. However, fine-tuning is expensive and can lead to catastrophic forgetting, a phenomenon where the model loses previously acquired knowledge during training on new data [2]. Retrieval-augmented

generation (RAG) [5] provides a more efficient alternative by retrieving relevant, up-to-date information from external sources during inference. This retrieval step grounds model outputs in factual evidence and extends the effective knowledge of LLMs beyond their static training data. Section 2.2 provides an overview of information retrieval, which serves as the basis for incorporating external knowledge into language models via RAG, as discussed further in Section 2.4.

2.2 Information Retrieval

Information retrieval plays a central role in RAG by providing LLMs with relevant external knowledge that grounds their responses in factual content. Retrieval methods are commonly divided into sparse and dense approaches, which differ in how they represent text as vectors [43]. These approaches are often referred to as keyword search and semantic search, respectively.

Sparse representations use high-dimensional vectors whose length equals the vocabulary size, with most entries being zero since each document contains only a small subset of all possible words. Dense representations, in contrast, encode text into a lower-dimensional continuous space where semantic information is distributed across the vector dimensions. Section 2.2.1 presents sparse retrieval methods, while Section 2.2.2 discusses dense retrieval.

2.2.1 Sparse Retrieval

Sparse retrieval relies on exact term matching between a query and documents, representing text as a high-dimensional sparse vector of word occurrences [43]. Over time, methods such as bag-of-words, TF-IDF, and BM25 have progressively improved term weighting to better approximate document relevance.

Bag-of-Words The bag-of-words approach is a straightforward way to represent text in numerical format [44]. In this model, a document is represented as a vector of word counts, where each position in the vector corresponds to a unique word in the vocabulary. This approach ignores word order and focuses only on how often each word appears in the document. These counts are known as term frequencies, and the number of occurrences of term t in document d is denoted by $\text{tf}_{t,d}$.

While simple to understand, the bag-of-words model has a major limitation: it treats all terms equally. This means common words receive the same weight as more informative terms, even though they contribute little to distinguishing between documents.

TF-IDF TF-IDF enhances the bag-of-words model by down-weighting common terms and up-weighting informative terms [44]. It does so by scaling the raw term frequencies by a factor called inverse document frequency.

The inverse document frequency of term t is defined as:

$$\text{idf}_t = \log \frac{N}{\text{df}_t},$$

where df_t is the number of documents in the collection that contain a term t and N is the total number of documents in a collection. The complete TF-IDF weighting for term t in document d is given by:

$$\text{tf-idf}_{t,d} = \text{tf}_{t,d} \cdot \text{idf}_t.$$

To compute the relevance of a document to a query q, we can sum the TF-IDF scores of all query terms present in the document:

$$\text{TF-IDF}(q, d) = \sum_{t \in q} \text{tf-idf}_{t,d}.$$

While TF-IDF effectively reduces the influence of common words, it still has limitations. The linear scaling of term frequency can overweight repeated terms, and the absence of document length normalization can bias results toward longer documents.

BM25 The BM25 family of ranking functions [45] refines TF-IDF by introducing non-linear term frequency scaling and explicit document length normalization. These enhancements make BM25 more robust to variations in document length and term repetition. The BM25 score for a document d with respect to a query q is defined as:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{idf}_t \cdot \frac{\text{tf}_{t,d} \cdot (k_1 + 1)}{\text{tf}_{t,d} + k_1 \cdot (1 - b + b \cdot \frac{|d|}{\text{avgdl}})},$$

where $\text{tf}_{t,d}$ is the frequency of term t in document d, $|d|$ is the length of the document d and avgdl is the average document length in the corpus. k_1 and b are tunable parameters and the value of k_1 is usually chosen between 1.2 and 2 and b is set to 0.75 in the absence of advanced optimization [44].

The IDF component in BM25 is usually computed as:

$$\text{idf}_t = \log \left(\frac{N - \text{df}_t + 0.5}{\text{df}_t + 0.5} \right),$$

where df_t is the number of documents containing the term t and N is the total number of documents in the collection, as before.

By using a saturating function on term frequency and adjusting for document length, BM25 yields more balanced relevance scores compared to TF-IDF. These enhancements make BM25 the foundation for many modern text-ranking methods in both academic research and commercial applications [43].

Although term-weighting models can estimate term importance from the statistics of texts, exact-match approaches fail completely when query terms do not appear in the documents at all [43]. This limitation motivates the adoption of dense retrieval techniques, discussed below.

2.2.2 Dense Retrieval

Dense vector representations, or embeddings, map words or sentences into fixed-length numerical vectors that capture their semantic meaning. Words with similar meanings are embedded close to one another in the vector space, enabling semantic relationships to be measured geometrically. For instance, the well-known example vec("king") − vec("man") + vec("woman") ≈ vec("queen") illustrates how vector arithmetic can capture semantic relationships between words [46].

Before the advent of transformer-based models, popular approaches for generating word embeddings included word2vec [47] and GloVe [48]. However, these methods produced static embeddings that did not account for the surrounding context, assigning identical representations to a word regardless of its meaning across different sentences. Consequently, they struggled to represent polysemous words and to capture deeper contextual nuances [49].

The introduction of transformer architectures, such as BERT [13], enabled the development of contextual embeddings, in which the vector representation of a word depends on its surrounding words [49]. This allows models to differentiate between different meanings of the same word, improving performance in tasks that require semantic understanding.

The idea of embeddings can be generalized to the sentence level, resulting in sentence embeddings. A straightforward approach is to input a sentence into BERT and extract the embedding of the special `[CLS]` token or average the output token vectors. However, these approaches have been shown to perform often worse than averaging GloVe embeddings for tasks such as semantic textual similarity [50].

BERT achieves strong results as a cross-encoder, where a pair of sentences is jointly processed to predict their similarity [43]. In this setup, the entire sentence pair is passed through the model, enabling fine-grained interactions between the two sentences. However, this approach is computationally expensive. For example, computing pairwise similarity between 10,000 sentences requires approximately 50 million inference computations and can take up to 65 hours [50]. This makes cross-encoders impractical for large-scale retrieval.

A more efficient alternative is the bi-encoder architecture, where each input is encoded independently into its own vector [43]. These embeddings can be pre-computed and indexed, enabling rapid retrieval based on vector similarity. While this method is much faster, it generally sacrifices some accuracy compared to cross-encoders. Figure 4 illustrates the architectural difference between cross-encoder and bi-encoder approaches.

Sentence-BERT (SBERT) [50] implements this bi-encoder approach using a siamese network architecture. Each sentence is encoded independently into a fixed-length vector, allowing rapid comparison via cosine similarity. This approach reduces the time required to compute all pairwise similarities among 10,000 sentences from around 65 hours to just 5 seconds while maintaining competitive accuracy [50].

Building on the bi-encoder architecture, the Dense Passage Retriever (DPR) [51] adapts it for open-domain question answering, enabling efficient retrieval of relevant passages from large text corpora. While SBERT uses a siamese architecture in which

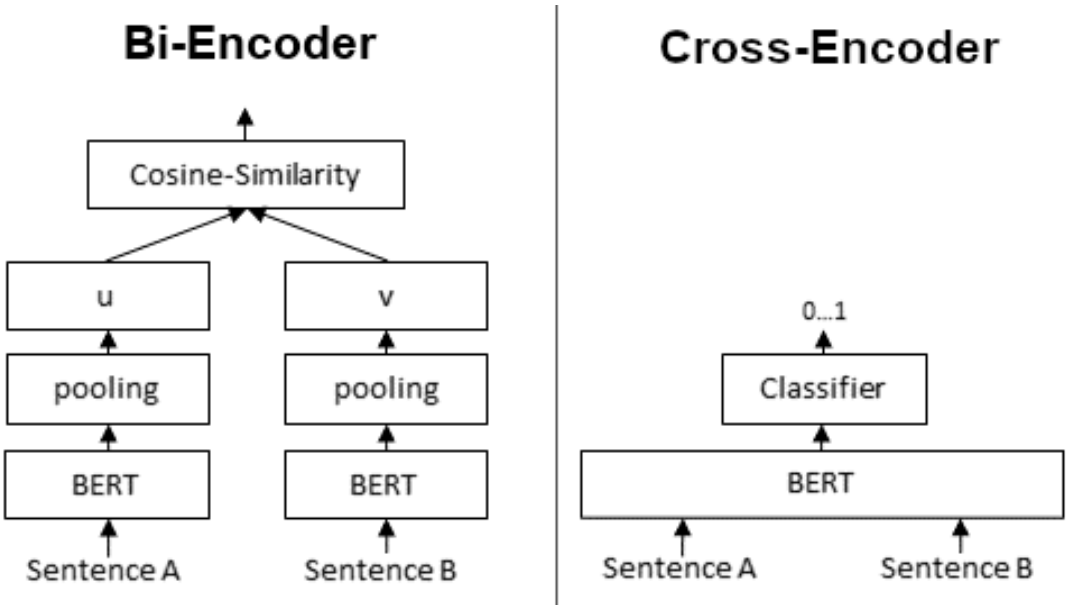


Figure 4: Comparison between cross-encoder and bi-encoder architectures [50]. In the cross-encoder, both input sentences are processed jointly by a transformer to produce a similarity score, yielding high accuracy but limited scalability. The bi-encoder encodes each input independently, allowing pre-computed embeddings and efficient similarity search at the cost of reduced precision.

both encoders share weights, DPR employs two distinct BERT-based encoders: one for questions and another for passages. This dual-encoder setup allows DPR to optimize representations specifically for the question-answering task. The experiments done in [51] show that DPR outperforms BM25 on a wide range of open-domain QA datasets. DPR is also utilized in the original RAG framework, as discussed in Section 2.4.1.

ColBERT (Contextualized Late Interaction over BERT) [52] bridges the efficiency-quality trade-off between bi-encoders and cross-encoders through a late interaction architecture. Rather than compressing queries and documents into single vectors, ColBERT independently encodes them into matrices of token-level embeddings. Similarity is computed using the MaxSim operator: each query token finds its best-matching document token via cosine similarity, and these maximum similarities are summed for the final relevance score. This token-level interaction captures richer semantic matches than single-vector approaches while maintaining efficiency through pre-computed document embeddings.

While dense retrieval models effectively encode semantic similarity between queries and documents, their practical deployment requires efficient methods for searching large-scale embedding spaces. Nearest neighbor search techniques provide a scalable solution to this problem.

2.3 Nearest Neighbor Search

In dense retrieval, finding relevant documents can be formulated as a nearest neighbor search problem, where the goal is to identify the document vectors most similar to a given query vector with respect to a similarity metric [43]. A straightforward approach is to compare the query vector with all document vectors using a brute-force search. However, this method quickly becomes inefficient and impractical as the size of the document collection increases.

Scalable solutions to the nearest neighbor search problem rely on approximate nearest neighbor search (ANNS), which offers a practical trade-off between efficiency and precision. This trade-off is acceptable in practice because similarity metrics serve only as proxies of relevance and do not perfectly model the task to begin with [43]. Faiss [53] is a popular open-source library that provides efficient implementations of various ANNS algorithms. Moreover, ANNS is a key feature of modern vector databases [54], which are widely used in RAG systems.

ANNS algorithms can be broadly categorized into table-based, tree-based, and graph-based indexing methods [54]. One widely adopted and effective method for ANNS is the Hierarchical Navigable Small World (HNSW) algorithm [55], a graph-based technique, which is one of the state-of-the-art methods for approximate nearest neighbor search [56].

2.3.1 Hierarchical Navigable Small World

Hierarchical Navigable Small World (HNSW) [55] is a highly efficient ANNS algorithm. HNSW organizes data points into a multi-layer graph structure that enables fast and scalable searches by combining principles from navigable small-world graphs and hierarchical designs inspired by probabilistic skip lists. Each layer of the graph contains a subset of the nodes, with higher layers including progressively fewer nodes. The nodes are connected to their nearest neighbors by edges, creating a small-world network in which most nodes can be reached through only a few steps, and groups of neighboring nodes tend to be densely interconnected. This organization enables the graph to be traversed quickly, enabling the discovery of approximate nearest neighbors without exhaustively examining all data points. Figure 5 illustrates the multi-layer graph structure of HNSW.

The search process in HNSW begins at the highest layer of the hierarchy, where the graph is sparsest. Starting from a random entry point, the algorithm performs a greedy search by repeatedly moving to the neighbor closest to the query point until it reaches a local minimum in that layer. Once this local minimum is found, the search proceeds to the next level, using the found node as the starting point for the next layer. This process continues until the bottom layer is reached, which contains all nodes in the dataset and has the densest connectivity. The local minimum found in the bottom layer serves as the approximate nearest neighbor for the query. The red arrows in Figure 5 illustrate the hierarchical search process.

The index construction in HNSW is incremental and closely mirrors the search procedure. The algorithm first determines the maximum layer in which the new node

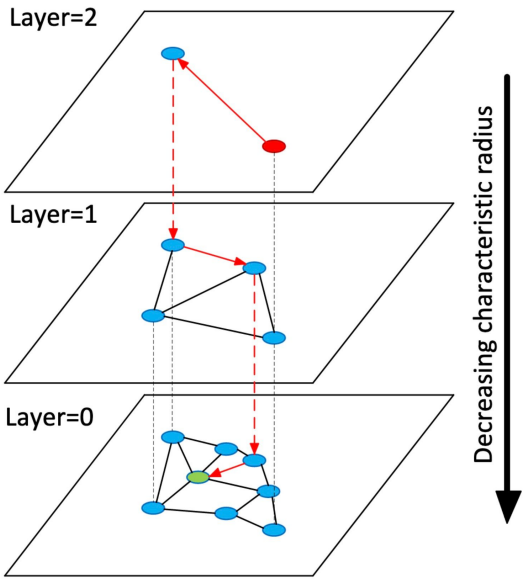


Figure 5: Hierarchical Navigable Small World (HNSW) graph structure [55]. HNSW organizes data points into multiple graph layers, where higher layers contain fewer nodes with long-range connections and lower layers provide dense local neighborhoods. During search, the algorithm traverses from upper to lower layers, gradually narrowing the search radius to efficiently locate approximate nearest neighbors. The red arrows illustrate the hierarchical descent from coarse to fine search.

will appear, selecting it randomly with exponentially decreasing probability for higher layers. Insertion begins at the top layer of the existing graph and proceeds downward, using the search procedure to locate the region where the new node is to be inserted. Once the algorithm reaches the node's assigned maximum layer, it connects the new node to a limited number of its nearest neighbors within that layer and all lower layers. Throughout construction, connections are maintained to preserve the small-world properties of the graph, ensuring efficient navigation and scalability to large datasets.

Efficient ANNS methods, such as HNSW, enable the scalable retrieval of relevant documents from large vector databases. These retrieval capabilities form the foundation for RAG systems, where retrieved information is combined with LLMs to produce grounded responses.

2.4 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [5] is a technique that enhances LLMs by integrating external knowledge sources. Instead of relying solely on information encoded in the model's parameters, often referred to as parametric memory, RAG incorporates a non-parametric memory, such as an external database or document collection, to supplement the model's responses. By grounding generation in retrieved context, RAG has been shown to reduce hallucinations [57] and to provide a more effective means of introducing new knowledge than fine-tuning [58].

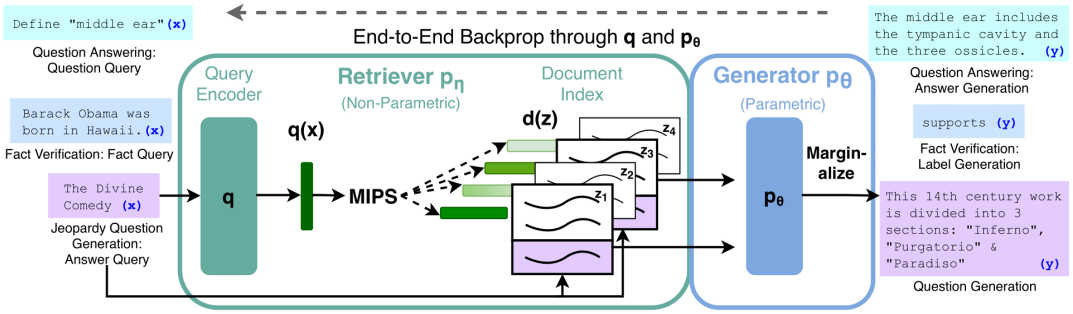


Figure 6: Architecture of the original retrieval-augmented generation (RAG) system [5]. The framework integrates a non-parametric retriever based on DPR, with a parametric generator based on BART. The retriever encodes the query, retrieves relevant documents from an index, and passes them to the generator, which conditions its output on both the retrieved context and its internal parameters. This enables end-to-end training that combines retrieval and generation.

Early RAG research primarily focused on training specialized language models with integrated retrieval components [5, 59, 60]. However, following the emergence of powerful in-context learning capabilities [18], recent approaches have shifted toward incorporating retrieved information directly into the model prompt [61, 62, 63]. This shift enables RAG to be applied to models accessible only through APIs. We briefly review the original RAG framework in Section 2.4.1 and examine modern RAG paradigms in more detail in Section 2.4.2.

2.4.1 Original RAG Framework

The original RAG framework, which combines pre-trained parametric and non-parametric memory for language generation, was introduced by Lewis et al. [5] in 2020. The RAG system consists of two components: a retriever based on DPR [51] and a generator based on BART [23]. Figure 6 presents a high-level overview of the approach.

The system takes an input sequence x and retrieves a set of text documents z that serve as contextual information during the generation of the output sequence y. The retriever, denoted $p_\eta(z \mid x)$ with parameters η, returns the top k most relevant text passages given the query x. The generator, $p_\theta(y_i \mid x, z, y_{1:i-1})$ parameterized by θ, predicts the current token y_i based on the preceding tokens $y_{1:i-1}$, the input x, and the retrieved document z. Together, these components are combined into a probabilistic model that is trained end-to-end by treating the retrieved document as a latent variable.

Lewis et al. [5] propose two approaches, RAG-Sequence and RAG-Token, which differ in how they marginalize over retrieved documents to generate text. In RAG-Sequence, the model conditions the entire output sequence on the same retrieved document, while RAG-Token allows each generated token to be conditioned on potentially different documents. The authors fine-tune and evaluate the RAG models on various knowledge-intensive NLP tasks, outperforming models that rely solely on parametric memory. From these initial formulations, the RAG framework has since

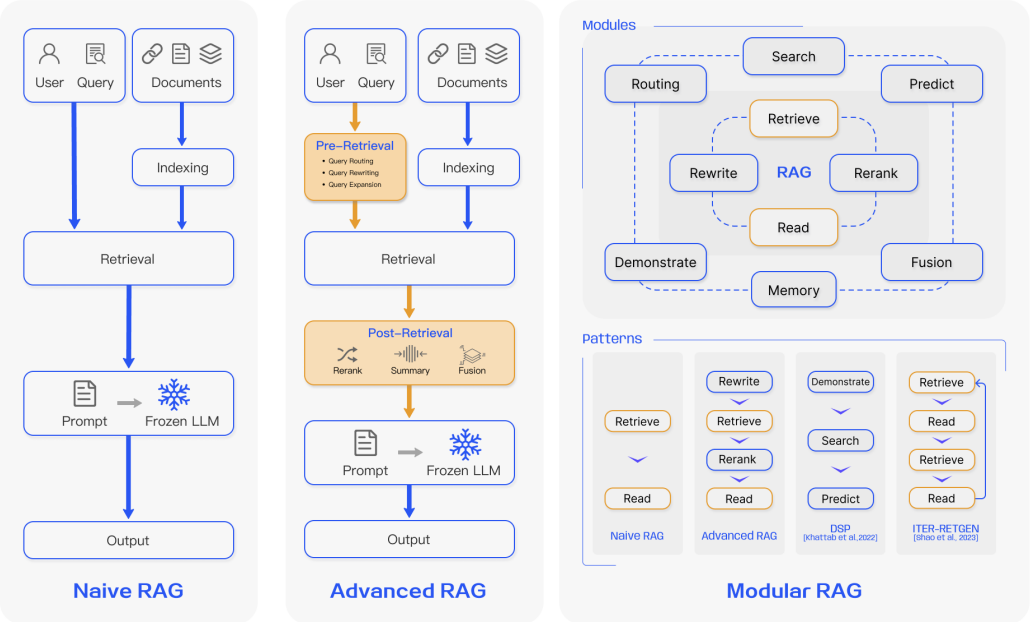


Figure 7: Overview of RAG paradigms [6]. Naive RAG performs direct retrieval followed by generation. Advanced RAG introduces pre- and post-retrieval steps such as query rewriting, reranking, and summarization. Modular RAG abstracts these components into flexible modules that can be composed into more complex pipelines.

developed into several modern paradigms.

2.4.2 Modern RAG Paradigms

Modern RAG paradigms can be broadly categorized based on how the retrieval and generation components have evolved since the original framework was introduced. Gao et al. [6] identify three primary paradigms: naive RAG, advanced RAG, and modular RAG, which reflect increasing levels of sophistication and flexibility in the RAG pipeline. Figure 7 presents an overview and comparison of these paradigms, which are described in more detail below.

Naive RAG A naive RAG pipeline typically consists of three stages: indexing, retrieval, and generation. Figure 8 illustrates how these stages work together to answer a user's question.

The indexing stage begins by splitting a collection of documents into smaller chunks, making them suitable for the limited context window of LLMs. The source documents can be in various formats, such as PDF, HTML, or Markdown. Each chunk is then transformed into a vector representation using an embedding model and stored in a vector database.

During the retrieval stage, when a user submits a question, the query is embedded using the same embedding model employed during indexing. A similarity score is

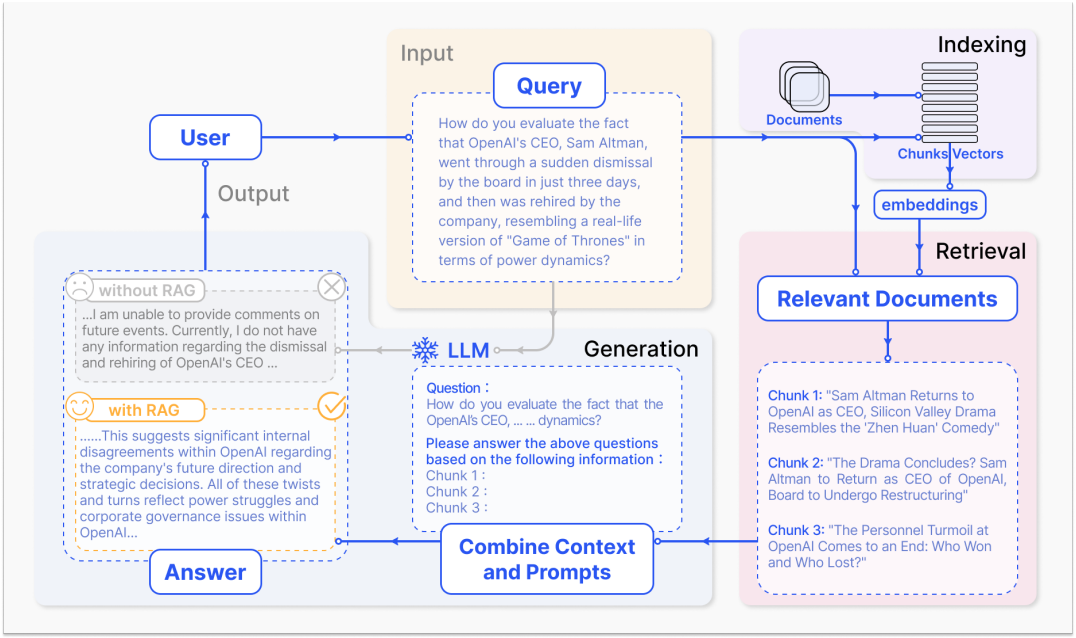


Figure 8: Naive RAG process [6]. The pipeline consists of three main stages: indexing, where documents are chunked and embedded into a vector database, retrieval, where relevant chunks are selected based on similarity to the query, and generation, where the retrieved context and user query are combined to produce the final answer.

computed between the embedded query and the stored document embeddings, and the top k most relevant chunks are then retrieved based on this score.

Finally, in the generation stage, the user's query and the retrieved context are combined into a prompt, which is passed to the LLM. The model then generates the final answer for the user.

Due to its simplicity, the naive RAG approach has several limitations [6]. The retrieval step may return misaligned or irrelevant chunks, leading to incorrect or unhelpful responses. The LLM may also hallucinate, particularly if it struggles to effectively utilize the retrieved context. In addition, responses can become repetitive if similar information from multiple sources is included in the prompt. There is also a risk that the LLM will overly rely on the retrieved content, producing outputs that simply repeat the retrieved information without any meaningful synthesis.

Advanced RAG Advanced RAG employs pre- and post-retrieval strategies to improve retrieval quality and mitigate the limitations of naive RAG pipelines. The focus in the pre-retrieval stage is on optimizing both the indexing structure and the original user query. Additionally, hybrid retrieval, which combines keyword and semantic search, can further improve retrieval quality by combining the strengths of different retrieval methods.

Indexing optimization aims to enhance the quality of the indexed content. A central element of this process is the chunking strategy, which defines how documents are divided into smaller, retrievable units. A popular approach is to divide the text into

chunks of a fixed token length. Another method is to control the granularity by splitting the text into meaningful segments, such as sentences or paragraphs. In addition to refining the chunking strategy, each chunk can be supplemented with metadata such as the author, category, or timestamp to provide useful contextual information during retrieval.

Query optimization focuses on adapting the user's query to improve retrieval quality. Common techniques include query expansion, query transformation, and query routing. In query expansion, the original query is expanded into multiple variations, enriching its content and providing additional context to increase the likelihood of retrieving relevant information. In query transformation, the original query is rewritten to better align with the structure and language of the indexed content, improving retrieval accuracy. Finally, query routing leverages metadata attached to chunks to narrow the search scope, allowing the retrieval process to focus on the most relevant subset of the data.

The purpose of post-retrieval processing is to effectively combine the retrieved context with the user's query, ensuring the model generates a high-quality response. The main techniques used at this stage are reranking and context compression. Reranking rearranges the retrieved content within the prompt, placing the most relevant information at the beginning. This is motivated by the observation that LLMs struggle to effectively utilize information in the middle of long prompts [64]. Simply appending all retrieved content to the prompt can result in information overload, causing the model to focus on irrelevant or redundant details. To address this, context compression aims to distill retrieved context by selecting and highlighting only the most essential information, thereby reducing prompt length and ensuring that critical content receives the model's attention.

Modular RAG Modular RAG evolves beyond naive and advanced RAG by adopting a more flexible and composable architecture. Instead of relying on a fixed retrieval and generation pipeline, it introduces interchangeable modules that can be configured for different tasks and data sources, allowing it to handle diverse retrieval scenarios and processing needs.

In addition to adding new modules, modular RAG also supports more flexible pipeline patterns. Instead of the simple retrieve-then-read process used in naive and advanced RAG, modular RAG systems can perform iterative retrieval, repeatedly searching the knowledge base based on the initial query and the text generated so far. They can also employ recursive retrieval, which improves search queries by leveraging the outcomes of previous retrieval steps, or adaptive retrieval, which dynamically determines when and what to retrieve during the generation process. This flexibility enables the architecture to support diverse use cases and allows individual components, such as the retriever or generator, to be fine-tuned for improved performance.

The increasing flexibility and complexity of RAG architectures highlight the need for systematic evaluation methods. Understanding how each module contributes to overall performance is therefore essential, as discussed in the following section.

2.5 Evaluation of Retrieval-Augmented Generation

This section reviews the main challenges and approaches for evaluating retrieval-augmented generation. We discuss key evaluation challenges, dimensions, and metrics, followed by a discussion of the LLM-as-a-judge paradigm. The section concludes with the Ragas framework used for automated evaluation in this thesis.

2.5.1 Evaluation Challenges

Evaluating RAG systems requires a comprehensive approach that considers both the retrieval and generation components, as well as the system as a whole [65]. Each of these components presents distinct challenges.

On the retrieval side, evaluation is complicated by the vast and dynamic nature of knowledge sources. The relevance and accuracy of retrieved content can change over time, and traditional metrics such as precision and recall often fail to capture the nuances of retrieval quality in RAG. Moreover, the risk of retrieving misleading or low-quality information calls for more sophisticated, task-specific evaluation methods.

On the generation side, the focus is on how accurately the output reflects both the retrieved content and the original query. This involves measuring factual accuracy, relevance, and coherence, factors that can be difficult to quantify, especially in open-ended question answering, where correctness is subjective.

However, evaluating a RAG system holistically is more than the sum of its parts [65]. The interaction between retrieval and generation must also be assessed, particularly how effectively the system leverages the retrieved content to enhance the quality of its responses. Additionally, practical factors such as response time and the system's ability to handle unclear or complex queries play an important role in determining overall system performance and usability.

2.5.2 Evaluation Aspects

Evaluating RAG systems involves assessing multiple aspects that capture the quality of both retrieval and generation [65]. In retrieval, relevance measures how well the retrieved documents match the intent of the query, while accuracy reflects the system's ability to prioritize relevant documents over irrelevant ones. For generation, relevance captures alignment with the query, faithfulness evaluates whether the response accurately reflects the retrieved content, and correctness assesses factual consistency with the ground-truth answer.

Another dimension of RAG evaluation focuses on the fundamental abilities that effective systems should demonstrate [66]. This includes four key evaluation tasks. Noise robustness measures the ability to extract useful information from partially relevant or noisy documents. Negative rejection evaluates whether the system can refrain from answering when no relevant information is available. Information integration tests the capacity to synthesize evidence from multiple documents, while counterfactual robustness assesses how well the system can identify and disregard misinformation when explicitly warned about it.

2.5.3 Evaluation Metrics

Retrieval Metrics Evaluating retrieval quality involves measuring how effectively a system returns relevant, accurate, and diverse information in response to user queries [65]. Metrics must capture both precision in identifying relevant content and robustness in handling large and dynamic information sources.

Retrieval metrics can be broadly categorized into non-rank-based and rank-based approaches [65]. Non-rank-based metrics assess relevance without considering position in the result list. Common examples include accuracy, which measures the proportion of correct outcomes, precision, which evaluates the fraction of retrieved items that are relevant, and recall@k, which measures the proportion of relevant items among the top k results. Rank-based metrics account for the ordering of results, emphasizing the importance of presenting relevant items first. Popular metrics in this category include Mean Reciprocal Rank (MRR) and Mean Average Precision (MAP).

Generation Metrics Human evaluation remains the gold standard for assessing text generation quality [65]. However, it is often expensive, time-consuming, and subject to variability across evaluators or time [10, 67].

To overcome these challenges, automatic evaluation offers a more scalable alternative. Traditional metrics such as BLEU [68] and ROUGE [69] rely on n-gram overlap between generated and reference texts. While widely used, they often fail to capture semantic similarity and correlate weakly with human judgments [10].

BERTScore [70] provides a more semantically informed measure by leveraging contextual embeddings from BERT to assess similarity between generated and reference texts. Unlike n-gram-based metrics, BERTScore considers word meaning in context, making it more robust to paraphrasing and better aligned with human evaluations.

Recently, the LLM-as-a-judge paradigm [11] has emerged as a promising approach, leveraging the reasoning and comprehension abilities of large language models to provide richer, context-aware evaluations.

2.5.4 LLM-as-a-Judge

LLMs can act as evaluators by leveraging their in-context learning capabilities, guided through structured prompts and examples. Several prompting paradigms are commonly used for this purpose, including scoring, true/false judgments, pairwise comparisons, and multiple-choice selection [71].

In scoring, the model assigns a numeric value to an answer based on criteria such as relevance, helpfulness, or factual accuracy. True/false judgments involve verifying whether a statement or answer is correct. Pairwise comparisons ask the model to select the better of two candidate responses, while multiple-choice selection requires choosing the most suitable option from a predefined set.

A common approach is to employ a powerful, general-purpose LLM as the evaluator [71]. Empirical studies have shown that advanced models such as GPT-4 can reach over 80% agreement with human judgments, comparable to inter-human agreement levels [11]. Further research has demonstrated strong alignment between

human and LLM evaluations across identical samples and instructions, indicating that LLM-based evaluation offers greater reproducibility and efficiency than traditional human assessment [72].

Despite these advantages, the reliability of this method depends strongly on the evaluator model's ability to follow instructions and reason effectively [71]. Several biases have also been observed in LLM-based evaluation, including position bias, where the model tends to favor responses that appear first, verbosity bias, where longer answers are preferred even if they are less accurate, and self-enhancement bias, where the model favors responses it has generated itself [11]. Mitigation strategies such as response order swapping, few-shot prompting, and reference-guided evaluation, where the LLM first generates its own answer to serve as a reference, can improve consistency and reduce these biases.

Building on these developments, automated evaluation frameworks such as Ragas [12] and ARES [73] have been introduced, leveraging LLM-based evaluators to assess both retrieval and generation in RAG pipelines.

2.5.5 Ragas

Ragas (Retrieval-Augmented Generation Assessment) [12] is an automated framework for evaluating RAG systems. It measures both the retrieval and generation components independently to provide a detailed understanding of system performance.

In its initial version, Es et al. [12] proposed three reference-free metrics that do not rely on ground-truth annotations: faithfulness and answer relevance for generation, and context relevance for retrieval. Faithfulness assesses whether the generated answer is grounded in the retrieved context, while answer relevance measures how directly the answer addresses the question. Context relevance evaluates the focus of the retrieved passages, favoring concise, relevant information.

To validate the framework, the authors constructed a new dataset called WikiEval, consisting of (question, context, answer) triplets derived from 50 Wikipedia pages covering events since early 2022. Each instance was annotated by two human evaluators across the three quality dimensions. Ragas metrics were then compared against human preferences in pairwise evaluations, achieving 95% alignment for faithfulness, 78% for answer relevance, and 70% for context relevance, outperforming other GPT-based evaluation methods.

Since its release, Ragas has developed into a comprehensive framework offering a broad set of metrics for evaluating LLM-based systems and automating test data creation. This thesis applies a selection of these metrics to capture retrieval effectiveness, answer quality, and overall end-to-end performance, which are formally defined in Section 3.5.1.

3 Methods

This section describes the methodology used to address the three research questions introduced in Section 1. We developed a retrieval-augmented generation (RAG) system and evaluated how its individual components influenced performance. The study followed a stepwise optimization process: starting from a simple baseline, we progressively refined document parsing and chunking, embeddings, retrieval strategies, and LLMs. This modular design allowed us to isolate the effect of each component on the retrieval and generation quality.

During optimization, we evaluated both open-source and proprietary models to determine whether smaller, self-hostable alternatives could approach the performance of models accessible only via API. This comparison spanned both embedding models and LLMs, providing insight into how model choice affected RAG effectiveness. To keep the scope manageable, we focused on models that could be run on resource-constrained local hardware, were widely adopted within research and developer communities, and were released by established organizations rather than individuals or small groups. Only instruction-tuned LLMs were included, as they are designed to follow prompts and generate relevant responses. Because the research paper dataset was in English, multilingual capabilities were not considered.

Model selection was guided by public leaderboards: the Massive Text Embedding Benchmark (MTEB) [7] for embeddings, and the Open LLM Leaderboard [8] and Chatbot Arena [9] for LLMs. While these benchmarks served as useful reference points, a key aim of this work was to examine how closely leaderboard performance corresponded to results in a real-world RAG setting. All configurations were evaluated using Ragas [12], a framework based on the LLM-as-a-judge paradigm that enables the automated assessment of both retrieval and generation quality, providing a reproducible and scalable basis for comparing system variants. To ensure that the system genuinely tested the retrieval of new information, we used recent research papers published after the knowledge cutoffs of all evaluated LLMs. The following sections describe the experimental setup, data collection, baseline system, RAG component optimizations, and evaluation procedure.

3.1 Experimental Setup

We implemented the RAG system using the LangChain framework [74], a widely adopted toolkit for building LLM-powered applications. LangChain provides flexible integration with various data sources, vector databases, embedding models, and LLMs, which makes it suitable for experimenting with different configurations in this thesis. To monitor and analyze the system's behavior, we used the LangSmith observability platform [75], which provided trace-level visibility into the RAG pipeline and helped examine intermediate inputs, outputs, and retrieved context during evaluation.

For vector storage and retrieval, we relied on Weaviate [76], an open-source vector database that supports hybrid search and uses the HNSW algorithm for indexing. Weaviate was deployed in a Docker container to ensure a consistent and isolated environment. Open-source models were accessed through the Sentence Transformers

library [50] for embeddings and through Ollama [77] for LLMs. These tools made it straightforward to test and switch between multiple models.

To support reproducibility, all experiments were conducted using LangChain version 0.2.16 and Weaviate version 1.32.4. Evaluation was performed using Ragas version 0.1.21. All experiments were run on an M2 MacBook Pro with 32 GB RAM, which constrained the size of models that could be executed locally.

In every experiment, we retrieved the top 4 documents using LangChain's default configuration. The only exception was the `MultiQueryRetriever` described in Section 3.4.4, where the number of retrieved documents could not be controlled. Retrieval was based on cosine distance, the default distance metric in Weaviate.

To ensure deterministic behavior and maximize reproducibility, we set the temperature to 0 for all LLM calls in this thesis. For OpenAI models, we also relied on model snapshots, which are fixed versions identified by date-based tags. Using snapshots prevents changes in the default API models from affecting our results over time.

GPT-4o [78] was used as the LLM for query optimization, LLM-based reranking, test set construction, and evaluation, with the snapshot `gpt-4o-2024-08-06`. The `text-embedding-3-large` model [79] was used as the embedding model for evaluation and test set construction. Both models were chosen for their strong performance and popularity at the time of writing. The prompt used across experiments is shown in Listing 1, adapted from the LangChain RAG tutorial [80].

Listing 1: Prompt template used in all RAG experiments. The model is instructed to answer questions based on the retrieved context and to respond with "I don't know" when the answer is not available.

```
You are an assistant for question-answering tasks. Use the
following pieces of retrieved context to answer the question.
If you don't know the answer, just say that you don't know.
Question: {question}
Context: {context}
Answer:
```

3.2 Data Collection

We used machine learning research papers from arXiv as the data source for our experiments. ArXiv is an open-access repository containing over two million scholarly articles across fields such as physics, mathematics, computer science, and related disciplines [81]. Machine learning papers were selected because they are openly accessible and often include complex structures, such as equations, tables, and figures, that present meaningful challenges for RAG systems. Furthermore, arXiv is frequently updated with new publications, allowing us to include recent papers that had not been part of the training data of LLMs. This aligns with one of the key advantages of RAG: the ability to incorporate up-to-date information that is not encoded in the parametric memory of the models.

To ensure a focus on high-quality papers, the selection targeted publications likely to contain significant findings or novel contributions. Papers with a high number of citations are often good indicators of quality. However, since arXiv receives a large volume of submissions daily and does not support filtering by citation count, identifying such papers manually was challenging. To address this, we turned to machine learning newsletters that highlight recent research and regularly feature noteworthy publications. Among these, Top ML Papers of the Week [82] provides a weekly curated list of influential AI and ML research maintained since early 2023. Ten papers are added to the list each week, most of which link to arXiv. The list is maintained in a single Markdown file on GitHub, enabling easy programmatic access.

We constructed our dataset from the GitHub-hosted Top ML Papers of the Week list by scraping the Markdown file to extract arXiv identifiers from paper links, excluding entries that did not point to arXiv. To keep the experiments feasible, we restricted the collection to papers listed between May 26 and June 29 in 2025. This decision was motivated by the substantial processing time required to parse the PDFs, as noted in Section 4.2.1. Using the extracted arXiv IDs, we retrieved the full papers through the arxiv.py library [83], a Python wrapper for the arXiv API.

The resulting dataset comprised 40 research papers in PDF format, totaling 1114 pages. Figure 9 shows the distribution of paper lengths. The papers ranged from approximately 10 to over 70 pages, with most falling within the 15-30 page range. The inclusion of both shorter and much longer works ensures that the dataset captures documents of varying depth and structure, providing a realistic basis for retrieval experiments.

Figure 10 presents the distribution of submission dates. The vast majority of the collected papers were submitted to arXiv between May and June 2025, corresponding to the period when they were featured in the newsletter. The only outlier was originally submitted in November 2024 but revised in April 2025, which likely explains its inclusion in the newsletter. This distribution highlights that the dataset mainly reflects very recent research. None of these papers should be included in the training data of the evaluated LLMs, since all models have knowledge cutoffs preceding the publication dates of the papers in our dataset, as discussed in Section 3.4.6.

3.3 Baseline RAG Pipeline

To establish a reference point for comparison, we began by implementing an initial RAG pipeline using commonly adopted defaults. This configuration corresponded to the naive RAG paradigm introduced in Section 2.4.2, which relied solely on dense retrieval without hybrid search, query optimization, or reranking.

In this baseline setup, research papers in PDF format were parsed using the pypdf library [84] and split into chunks with LangChain's `RecursiveCharacterTextSplitter`. We used a chunk size of 1024 characters with no overlap, aligning with best practices for initial RAG experiments [85].

The resulting text chunks were embedded using OpenAI's `text-embedding-ada-002` model [86], and responses were generated with GPT-3.5 Turbo [1] using the `gpt-3.5-turbo-0125` snapshot. Both models were widely adopted in early RAG

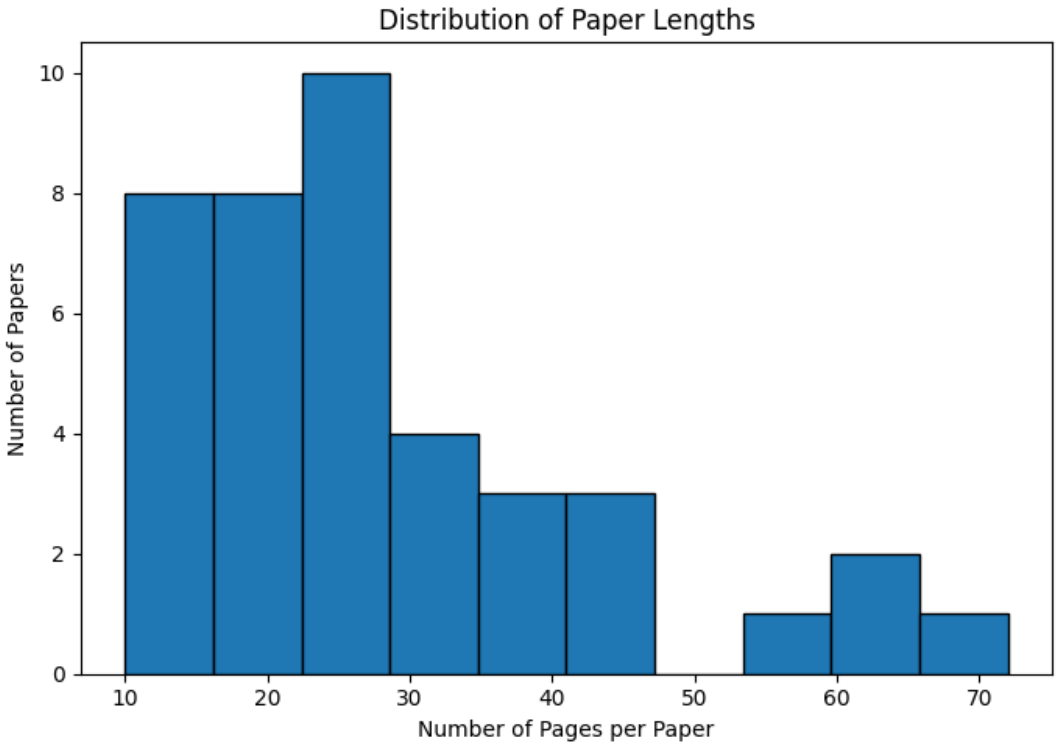


Figure 9: Distribution of paper lengths in the collected dataset.

systems, although more capable versions have since been released. Overall, this baseline represented a minimal viable RAG implementation that served as a foundation for evaluating the impact of the more advanced techniques introduced in subsequent sections.

3.4 RAG Pipeline Components

To improve on the baseline, we experimented with each component of the RAG pipeline individually. This modular approach allowed us to isolate the impact of different enhancements on overall performance. Specifically, we investigated alternative methods for document parsing and chunking, evaluated a diverse set of embedding models, and introduced several retrieval optimizations, including hybrid search, query optimization, and reranking. We also compared multiple LLMs for response generation. The following sections describe these components in detail, outlining their motivations and configurations.

3.4.1 Document Parsing and Chunking

The first step in our RAG pipeline was to convert the downloaded PDF files into text and split this text into smaller chunks that could be embedded and stored in our Weaviate vector database. LangChain offers various document loaders that support

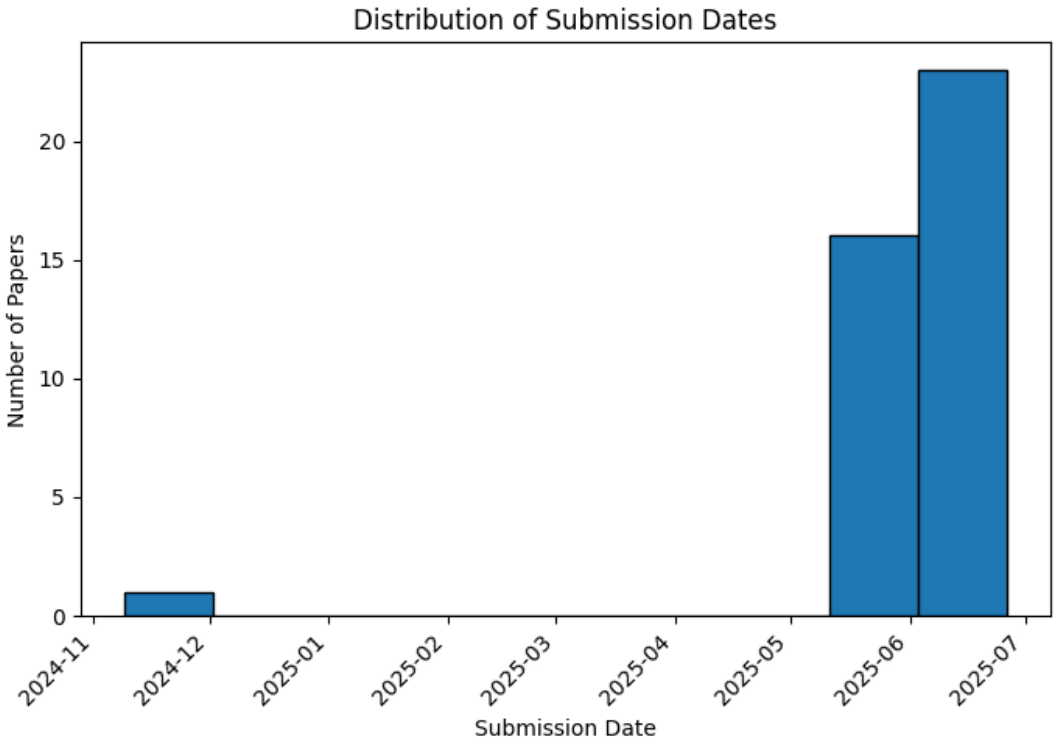


Figure 10: Distribution of arXiv submission dates for the collected research papers.

data ingestion from multiple sources and formats, extracting both the textual content and associated metadata for further processing.

We experimented with two different approaches for parsing and chunking the data. The first approach used the pypdf library [84] to parse the PDFs, followed by LangChain's `RecursiveCharacterTextSplitter` to divide the text into smaller chunks. The second approach used the Unstructured library [87], which handles both parsing and chunking. These two approaches are described in more detail below.

RecursiveCharacterTextSplitter In the first approach, PDF files were parsed into text using LangChain's `PyPDFLoader`, which leverages the pypdf library under the hood. This loader extracts the text from each page of the PDF individually.

The most straightforward way to split the text is by dividing it into fixed-size chunks of N characters, regardless of the content. Optionally, the chunks may include some overlap. However, this method is overly simplistic, as it can result in splits occurring mid-sentence or even within a word, which can negatively impact downstream processing.

A more robust approach is to split the text recursively using a prioritized list of separator characters. The process starts with the highest-priority separator, such as paragraph breaks, and attempts to divide the text. If any resulting chunk still exceeds the specified maximum length, the algorithm proceeds to the next separator in the list, such as line breaks or spaces. This continues until all chunks are within the desired size limit or no further separators remain. Compared to fixed-length chunking, this strategy

better preserves the semantic structure of the text and avoids splitting mid-sentence or mid-word.

We used LangChain's `RecursiveCharacterTextSplitter` for this task. By default, it applies the following list of separators, in order of priority: `["\n\n", "\n", " ", ""]`. The splitter also accepts configurable parameters for chunk size and chunk overlap, both defined in terms of character count.

The authors of [88] reported improved results when using the following list of separators instead of the default: `["\n\n", "\n", ".", "?", "!", " ", ""]`. We tested this configuration but did not observe a consistent or significant improvement in our setting. Therefore, we proceeded with the default separator configuration.

We varied the chunk size across three values: 512, 1024, and 2048 characters. These values correspond approximately to 128, 256, and 512 tokens, based on the common heuristic that one token roughly equals four characters in English [89]. To ensure comparability, the chosen chunk sizes match those used in [90]. The chunk overlap was defined as a percentage of the chunk size, and we tested three settings: 0%, 10%, and 20%. Before being passed to the `RecursiveCharacterTextSplitter`, the resulting overlap values were rounded to the nearest integer.

Unstructured We also explored using the Unstructured library for parsing and chunking PDF files via the `UnstructuredLoader` document loader provided by LangChain. Unstructured is an open-source library designed to convert unstructured content, such as PDF files or Word documents, into structured data. Although a hosted API version of Unstructured is available, we ran it locally to reduce costs.

Unlike `PyPDFLoader`, which extracts content page by page, Unstructured decomposes the input into semantically meaningful elements such as `Title`, `NarrativeText`, and `ListItem`, thereby preserving the document's logical structure.

Unstructured supports multiple strategies for partitioning content. Given the complex layout of scientific papers, we used a model-based `hi_res` partitioning strategy that leverages document layout information to more accurately identify structural components. We also evaluated the `fast` partitioning strategy, which relies on traditional NLP extraction techniques to quickly identify text elements. Although less sophisticated, it is significantly faster according to the Unstructured documentation.

The individual elements produced during the parsing step are not particularly useful on their own. Some may be too large to fit within the maximum input length of embedding models, while others, such as section titles, may be too small to provide meaningful context. To generate more informative and appropriately sized chunks, we used the chunking functionality provided by the Unstructured library. Unlike the `RecursiveCharacterTextSplitter`, which breaks text into smaller pieces, Unstructured groups consecutive text elements into chunks that are as large as possible without exceeding the `max_characters` limit. Elements are split only if their individual length exceeds the specified limit.

The output of this chunking process is a sequence of elements, each belonging to one of three types. All text content is returned as `CompositeElement` objects,

which may represent a single original element, a combination of multiple elements, or a fragment of a larger element that could not fit in one chunk. Tables are handled separately: those that fit within the size limit remain as `Table` elements, while larger tables are divided into `TableChunk` elements.

The Unstructured library offers several chunking strategies, including `basic`, `by_title`, `by_page`, and `by_similarity`. In this work, the `by_title` strategy was used, as it preserves the document's logical structure by treating each `Title` element as the start of a new section. When a `Title` element is encountered, the current chunk is closed and a new one is initiated, ensuring that each chunk contains content from only a single section. However, short paragraphs are occasionally misidentified as titles, leading to an excessive number of small chunks. To mitigate this issue, consecutive short sections that are likely misclassified are automatically merged into a single chunk, up to the specified `max_characters` limit. This modification maintains the document's structural coherence while producing more balanced and efficient chunks for downstream processing.

We used the same chunk sizes, 512, 1024, and 2048 characters, in the Unstructured approach as in the `RecursiveCharacterTextSplitter`. However, because the chunks were already segmented based on document structure and to reduce processing times, we did not apply any chunk overlap in the Unstructured approach.

Neither parsing approach, pypdf nor Unstructured, fully captured all document elements. Figures were not extracted, and tables and equations were only partially processed. As a result, the parsed data primarily represented the textual content of the papers, while visual and mathematical information was largely omitted.

3.4.2 Embedding Models

The next step in our RAG pipeline was to embed the document chunks. For this, we evaluated both proprietary models from OpenAI and open-source alternatives available via Hugging Face. From OpenAI, we included the older `text-embedding-ada-002` model [86] used in the baseline, as well as the newer `text-embedding-3-small` and `text-embedding-3-large` models [79].

In addition to these, we selected a set of open-source models of varying sizes from different organizations. To guide model selection, we referred to the Massive Text Embedding Benchmark (MTEB) leaderboard [7], which evaluates embedding models across tasks such as retrieval, clustering, and classification on diverse datasets. We aimed to examine how well the relative MTEB scores of our selected models aligned with downstream performance in our RAG use case. We also used the memory usage data provided in the leaderboard to assess whether each model was feasible to run on our hardware. Additionally, all selected models were required to support an input length of at least 512 tokens, which matches the largest chunk size used in our experiments.

The selected open-source models included three models from the BAAI General Embedding (BGE) family: `bge-small-en-v1.5`, `bge-base-en-v1.5`, and `bge-large-en-v1.5` [91], and two models from Alibaba's General Text Embeddings (GTE) series: `gte-base-en-v1.5` and `gte-large-en-v1.5` [92, 93]. These models

were chosen for their popularity, as indicated by high download counts on Hugging Face, and to enable comparison across different model sizes within the same architecture.

These embedding models are trained on English corpora using BERT-like architectures and are designed to produce general-purpose embeddings suitable for various tasks. When used for retrieval, the BGE models are optimized for use with instruction-style prompts. However, the v1.5 variants are less dependent on such inputs. In this study, they were employed in their default configuration without instruction prompts, as the LangChain framework does not provide a straightforward way to incorporate them.

Table 1 provides an overview of the selected models, including their number of parameters, embedding dimensionality, and maximum input length. Table 2 shows their corresponding MTEB scores: both the overall average across all tasks and the retrieval-specific average.

Table 1: Overview of embedding models evaluated in this study, including the number of parameters, embedding dimensionality, and maximum input length. Parameter counts for OpenAI models are not publicly available.

Model	Parameters (M)	Dimensionality	Max Tokens
text-embedding-ada-002	N/A	1536	8192
text-embedding-3-small	N/A	1536	8192
text-embedding-3-large	N/A	3072	8192
bge-small-en-v1.5	33.4	384	512
bge-base-en-v1.5	109	768	512
bge-large-en-v1.5	335	1024	512
gte-base-en-v1.5	137	768	8192
gte-large-en-v1.5	434	1024	8192

Table 2: MTEB scores for selected embedding models. The average score is computed across 56 datasets, while the retrieval score is averaged over 15 retrieval-specific datasets.

Model	MTEB Average Score	MTEB Retrieval Score
text-embedding-ada-002	60.99	49.25
text-embedding-3-small	62.26	51.08
text-embedding-3-large	64.52	55.43
bge-small-en-v1.5	61.76	51.71
bge-base-en-v1.5	63.19	53.28
bge-large-en-v1.5	63.89	54.34
gte-base-en-v1.5	64.11	54.09
gte-large-en-v1.5	65.39	57.91

3.4.3 Hybrid Search

To improve retrieval performance across a wide range of query types, we introduced hybrid search, which combines the strengths of sparse and dense retrieval methods [94]. Sparse vectors, used in traditional keyword-based retrieval, are effective for exact term matching, particularly useful in domain-specific queries. Dense vectors, on the other hand, capture the semantic meaning of queries, enabling more context-aware retrieval. By combining these approaches, hybrid search can handle both precise keyword queries and those that require a deeper understanding of language. This method is beneficial in real-world scenarios where user queries may contain ambiguous phrasing, misspellings, or specialized terminology, as it ensures that critical keywords are matched while also interpreting the broader intent behind the query.

In Weaviate, hybrid search is implemented by performing a sparse keyword search based on BM25 and a dense vector search in parallel [94]. The results from both searches are merged and reranked using a fusion algorithm to produce a single unified result list. The balance between keyword and vector search contributions is controlled by the `alpha` parameter in Weaviate: an `alpha` value of 0 corresponds to a pure keyword search, while an `alpha` value of 1 results in a pure vector-based search. When `alpha` is set to 0.5, an equal weight is given to both methods. In our experiments, we evaluated `alpha` values of 0, 0.25, 0.5, 0.75, and 1 to identify the balance that yielded the best retrieval performance.

3.4.4 Query Optimization

User-generated queries are often suboptimal, as they may contain irrelevant details or lack sufficient specificity. To address this, we explored query rewriting and query expansion techniques to better align queries with the user's intent. These techniques were implemented using the `RePhraseQueryRetriever` and `MultiQueryRetriever` retrievers from LangChain, which were selected for their ease of integration into the existing pipeline. Both approaches use an LLM to rewrite queries before retrieval. In our implementation, we used GPT-4o for this purpose, as described in Section 3.1. Listing 2 presents the default prompt used by `RePhraseQueryRetriever` to reformulate user queries.

Listing 2: Default prompt used by the `RePhraseQueryRetriever` to reformulate user queries into retrieval-oriented vector store queries.

```
You are an assistant tasked with taking a natural language
query from a user and converting it into a query for a
vectorstore. In this process, you strip out information that is
not relevant for the retrieval task.
Here is the user query: {question}
```

However, we observed that the model's output frequently included extraneous information beyond the reformulated query. To mitigate this, we slightly modified

the prompt to explicitly instruct the model to return only the rephrased query. The modified prompt used in our experiments is shown in Listing 3.

Listing 3: Modified prompt used in the `RePhraseQueryRetriever` to ensure the model outputs only the reformulated query without additional text.

```
You are an assistant tasked with taking a natural language
query from a user and converting it into a query for a
vectorstore. In this process, you strip out information that is
not relevant for the retrieval task. Return only the rephrased
query and nothing else.
Here is the user query: {question}
```

The `MultiQueryRetriever`, in contrast, generates multiple versions of the user's query from different perspectives. It then performs retrieval using each version and returns the union of all unique documents retrieved. This helps mitigate the issue where subtle variations in how a query is phrased can yield different retrieval results. We used the default prompt provided by LangChain without modification, as shown in Listing 4.

Listing 4: Default prompt used by the `MultiQueryRetriever` to generate multiple reformulations of a user query for improved document retrieval diversity.

```
You are an AI language model assistant. Your task is
to generate 3 different versions of the given user
question to retrieve relevant documents from a vector database.
By generating multiple perspectives on the user question,
your goal is to help the user overcome some of the limitations
of distance-based similarity search. Provide these alternative
questions separated by newlines.
Original question: {question}
```

To illustrate the difference between the two approaches, Table 3 shows an example query together with the outputs produced by `RePhraseQueryRetriever` and `MultiQueryRetriever`. The example highlights that while `RePhraseQueryRetriever` produces a single streamlined reformulation, `MultiQueryRetriever` generates diverse phrasings that capture slightly different perspectives on the original query.

3.4.5 Reranking

To improve retrieval performance in the post-retrieval phase, we employed reranking to ensure that the most relevant documents appeared at the top of the context window, where LLMs can utilize them most effectively [64]. We retrieved 50 documents to pass to the reranker, as this is a commonly used number in RAG systems [95], and selected the top 4 results, consistent with our other experiments.

Table 3: Example outputs from the `RePhraseQueryRetriever` and `MultiQueryRetriever` showing how user queries are reformulated to optimize retrieval performance.

Original Query	How does fine-tuning models impact their ability to abstain from providing incorrect answers?
RePhraseQueryRetriever Output	Fine-tuning models impact on abstaining from incorrect answers
MultiQueryRetriever Output	What effects does fine-tuning have on a model's capability to refrain from giving incorrect responses? In what ways does the process of fine-tuning influence a model's tendency to avoid incorrect answers? How does the fine-tuning of models affect their proficiency in withholding incorrect information?

We explored several reranking approaches. To identify suitable models, we searched for cross-encoders suitable for reranking on Hugging Face. We selected two models from the BGE family by BAAI: `bge-reranker-base` and `bge-reranker-large` [91], based on their popularity, as indicated by download counts, and their compatibility with our available hardware. These models are based on the XLM-RoBERTa architecture [96], a multilingual extension of RoBERTa, and they support English and Chinese. The base model contains 278 million parameters, while the large version has 560 million parameters.

To compare cross-encoder performance with alternative reranking strategies, we also evaluated ColBERTv2 [97], a storage-optimized version of ColBERT that retains the late interaction architecture. We integrated it using the RAGatouille library [98]. In addition, we experimented with reranking using an LLM through `LLMListwiseRerank` from LangChain, which is based on the implementation described in [99] and was configured to use GPT-4o as the underlying model, as described in Section 3.1.

Unlike pointwise rerankers such as cross-encoders, which evaluate individual query-document pairs independently, listwise rerankers take a query and a list of candidate documents as input and jointly produce a reordered list based on their relative relevance [99]. This allows the model to consider inter-document relationships, potentially leading to more coherent ranking decisions.

3.4.6 Large Language Models

After optimizing the retrieval pipeline, we evaluated how effectively different LLMs leveraged the retrieved context to generate accurate and coherent responses. To this

end, we compared a selection of both proprietary and open-source models.

Among the proprietary models, we included several developed by OpenAI, which are widely recognized for their strong general-purpose performance. To ensure consistent and reproducible results, we relied on the latest model snapshots available at the time of experimentation, rather than the default API versions, which may change over time. The evaluated models and their specific snapshots used in this thesis are listed in Table 4, although for readability we refer to them in the text by their common names: GPT-3.5 Turbo, GPT-4o, GPT-4o mini, GPT-4.1, GPT-4.1 mini, and GPT-4.1 nano.

Table 4: OpenAI language models and the exact model snapshots used in this thesis.

Model	Model Snapshot
GPT-3.5 Turbo	gpt-3.5-turbo-0125
GPT-4o	gpt-4o-2024-08-06
GPT-4o mini	gpt-4o-mini-2024-07-18
GPT-4.1	gpt-4.1-2025-04-14
GPT-4.1 mini	gpt-4.1-mini-2025-04-14
GPT-4.1 nano	gpt-4.1-nano-2025-04-14

In addition to proprietary models developed by OpenAI, we included a few recent open-source models released by different organizations. These models vary in size and were selected based on their recency and feasibility for running on our limited hardware. Specifically, we evaluated the Gemma 2 models with 9 billion and 2 billion parameters [30], the Llama 3.1 model with 8 billion parameters, and the Llama 3.2 models with 3 billion and 1 billion parameters [28].

To support and justify our model selection, we referred to two widely used evaluation resources: the Open LLM Leaderboard [8] and the Chatbot Arena [9]. The Open LLM Leaderboard presents average performance scores across six diverse benchmarks, designed to evaluate capabilities such as reasoning and general knowledge in both zero-shot and few-shot settings. The results are normalized on a scale where 0 represents random performance and 100 represents the theoretical maximum. Since the leaderboard includes only open-source models, it serves as a tool for comparing their relative performance but cannot be used to assess the performance of proprietary models.

The Chatbot Arena leaderboard ranks both proprietary and open-source LLMs based on crowdsourced human preferences. Scores are derived from aggregated pairwise comparisons, where users interact with a chat interface that presents responses from two anonymous models to the same prompt. After reviewing both outputs, the user selects the preferred response, and these votes are combined to compute each model's relative score.

Table 5 provides an overview of the selected models, including their context window sizes and scores from both the Open LLM Leaderboard and the Chatbot Arena.

In addition to model size and leaderboard performance, the knowledge cutoff date

Table 5: Selected language models with their context window sizes, average scores on the Open LLM Leaderboard, and Chatbot Arena scores. Proprietary models such as those from OpenAI are not available on the Open LLM Leaderboard.

Model	Context Window	Average Score	Arena Score
GPT-3.5 Turbo	16,385	N/A	1225
GPT-4o	128,000	N/A	1333
GPT-4o mini	128,000	N/A	1316
GPT-4.1	1,047,576	N/A	1409
GPT-4.1 mini	1,047,576	N/A	1379
GPT-4.1 nano	1,047,576	N/A	1321
Gemma 2 9B	8,192	32.07%	1265
Gemma 2 2B	8,192	17.05%	1202
Llama 3.1 8B	128,000	23.76%	1215
Llama 3.2 3B	128,000	24.20%	1172
Llama 3.2 1B	128,000	14.44%	1122

Table 6: Selected language models and their knowledge cutoff dates. Knowledge cutoff information for the Gemma 2 models is not publicly available.

Model	Knowledge Cutoff Date
GPT-3.5 Turbo	September 2021
GPT-4o	October 2023
GPT-4o mini	October 2023
GPT-4.1	June 2024
GPT-4.1 mini	June 2024
GPT-4.1 nano	June 2024
Gemma 2 9B	N/A
Gemma 2 2B	N/A
Llama 3.1 8B	December 2023
Llama 3.2 3B	December 2023
Llama 3.2 1B	December 2023

is another important characteristic. Table 6 lists these dates for all evaluated models. For most models, these dates are explicitly documented by their providers. In the case of Gemma 2, Google has not published a formal cutoff date. However, the release timeline shows that the 9B version was released in June 2024 and the 2B version in July 2024, suggesting that the training data did not extend beyond those months.

As shown in Figure 10, the collected dataset consists almost entirely of papers submitted to arXiv between May and June 2025, with one outlier submitted in November 2024. These submission dates are several months later than the knowledge cutoffs of all evaluated models. This ensures that none of the included papers were part of the models' training data, making the evaluation a genuine test of the RAG methods on unseen material.

3.5 Evaluation

We evaluated the performance of our RAG system using Ragas [12], which provides a range of LLM-based metrics for assessing retrieval and generation quality. From these, we selected six complementary metrics. Retrieval performance was measured using context precision and context recall, while answer generation quality was evaluated using faithfulness and answer relevancy. For end-to-end assessment, we applied answer semantic similarity and answer correctness, which compare the generated and ground-truth answers. Each metric is computed for every question individually, and the final score is the mean across all questions in the evaluated set. All metrics range between 0 and 1, with higher values indicating better performance.

3.5.1 Evaluation Metrics

Ragas evaluates RAG systems by combining LLM-based reasoning and semantic similarity measures to quantify different aspects of retrieval and answer quality. Below, we define the individual metrics used in this work.

Faithfulness Faithfulness measures the factual consistency of the generated answer with respect to the provided context. To compute this, a set of claims is first extracted from the generated answer. Each claim is then checked against the context to determine whether it can be inferred from it. The faithfulness score is defined as:

$$\text{Faithfulness} = \frac{|\text{claims in the generated answer supported by the context}|}{|\text{claims in the generated answer}|}.$$

Answer Relevancy Answer relevancy measures how well the generated answer addresses the original question. It is computed using the question, the context, and the answer. This metric does not assess factual correctness but penalizes answers that are incomplete or contain redundant information. The key idea is that if the answer is relevant and complete, an LLM should be able to reconstruct the original question from it. To capture this, multiple artificial questions are generated from the answer using an LLM, and the mean cosine similarity between their embeddings and the original question's embedding is computed:

$$\text{Answer Relevancy} = \frac{1}{N} \sum_{i=1}^{N} \cos(E_{g_i}, E_o),$$

where E_{g_i} is the embedding of the generated question i, E_o is the embedding of the original question, and N is the number of generated questions, which is 3 by default.

Context Precision Context precision measures the signal-to-noise ratio of the retrieved context by evaluating how well relevant chunks are ranked relative to less relevant ones. Ideally, the most relevant context items should appear near the top of the

retrieval list. This metric is computed using the question, ground truth, and retrieved contexts. Formally, it is defined as:

$$\text{Context Precision@K} = \frac{\sum_{k=1}^{K} (\text{precision@k} \times v_k)}{\text{total number of relevant items in the top } K \text{ results}}$$

$$\text{Precision@k} = \frac{\text{true positives@k}}{\text{true positives@k} + \text{false positives@k}},$$

where K is the number of chunks in contexts and $v_k \in \{0, 1\}$ is a binary indicator denoting whether the item at rank k is relevant.

Context Recall Context recall assesses how well the retrieved context covers the information needed to generate a complete and accurate answer. It is computed using the question, ground truth answer, and the retrieved context. To calculate context recall, each sentence in the ground truth answer is examined to determine whether it can be attributed to the retrieved context. Ideally, all claims in the ground truth should be supported by the context. The metric is defined as:

$$\text{Context Recall} = \frac{|\text{ground truth claims supported by context}|}{|\text{claims in ground truth}|}.$$

Answer Semantic Similarity Answer semantic similarity assesses the semantic resemblance between the generated answer and the ground-truth answer. It is computed as the cosine similarity between the embeddings of the two answers:

$$\text{Answer Semantic Similarity} = \cos(E_g, E_{gt}),$$

where E_g and E_{g_t} are embeddings of the generated and ground-truth answers, respectively.

Answer Correctness Answer correctness measures the accuracy of the generated answer compared to the ground truth. It combines factual similarity and answer semantic similarity through a weighted average. Factual similarity is determined by comparing factual statements between the generated and ground-truth answers. A statement is considered a true positive (TP) if it appears in both answers, a false positive (FP) if it appears only in the generated answer, and a false negative (FN) if it appears only in the ground-truth answer. Factual similarity is then quantified using the F1 score:

$$\text{F1 Score} = \frac{|\text{TP}|}{|\text{TP}| + 0.5 \times (|\text{FP}| + |\text{FN}|)}.$$

The final correctness score is computed as a weighted combination of factual similarity and semantic similarity, with default weights of 0.75 for factual similarity and 0.25 for semantic similarity.

3.5.2 QA Test Set Generation

To ensure reliable evaluation, we constructed a high-quality QA test set using the synthetic dataset generation functionality of Ragas. Each dataset entry consists of a question, its answer, and the related context. Ragas takes a novel approach to evaluation data generation by ensuring that the dataset reflects a variety of questions, including those of different difficulty levels. To do this, Ragas uses an evolutionary generation paradigm in which questions are systematically transformed to exhibit distinct characteristics. This includes creating reasoning questions that require logical inference and multi-context questions that demand synthesizing information from multiple chunks. By default, Ragas generates datasets containing 50% simple, 25% reasoning, and 25% multi-context questions.

Ragas relies on two collaborating LLMs: an actor, which generates questions and answers, and a critic, which evaluates and refines them. We used the GPT-4o model for both roles to ensure the generation of a high-quality test set, as described in Section 3.1. The embedding model was `text-embedding-3-large`. By default, Ragas processes chunks of up to 1024 tokens, automatically splitting longer ones. However, such splitting may occur mid-sentence, degrading the quality of generated questions. To avoid this, we parsed and chunked our PDFs using the Unstructured library with the `hi_res` partitioning strategy and the `by_title` chunking strategy and a chunk size of 4096 characters, roughly equivalent to 1024 tokens [89]. The resulting chunks were then used as inputs to Ragas. This ensures that chunk boundaries align with the semantic structure of the documents, eliminating the need for further splitting by Ragas.

A test set of approximately 100 QA pairs is commonly used in prior RAG evaluation studies [100, 101]. However, evaluating multiple configurations on such a large dataset would be prohibitively costly. We therefore adopted a two-stage evaluation strategy. The baseline and final optimized configurations were assessed using a larger set of 100 QA pairs, while intermediate experiments used a smaller set of 40 QA pairs. The two sets were entirely distinct, ensuring that the questions in the development set did not appear in the final evaluation set. This setup enabled cost-efficient iterative optimization while still allowing us to verify that improvements generalize to a larger, unseen dataset.

We initially generated a set of 300 (question, context, answer) triplets using Ragas with its default distribution of question types. Each question and its corresponding answer were then manually reviewed to ensure answerability and quality. During this two-stage filtering process, we first removed unanswerable questions and then discarded those with incomplete or uninformative answers. After filtering, the final dataset contained 213 questions: 107 simple, 54 reasoning, and 52 multi-context. Table 7 provides examples of questions that were excluded during this review process.

After filtering, we randomly sampled two non-overlapping datasets following the default Ragas distribution. The larger test set comprised 100 questions: 50 simple, 25 reasoning, and 25 multi-context, while the smaller development set contained 40 questions: 20 simple, 10 reasoning, and 10 multi-context. Examples from each question category in the test set are shown below.

Table 7: Example questions generated by Ragas that were removed due to low-quality or uninformative answers, or because they were unanswerable from the provided context.

Question	Answer	Reason for Removal
How can an agent effectively use tools to solve a task in a Web Browsing environment?	The answer to given question is not present in context	Question unanswerable from context
What issues come with classifying memory in LLMs?	The context does not explicitly mention issues with classifying memory in LLMs.	Question unanswerable from context
How do Qwen2.5 models of different scales perform on the sentencing prediction task in the legal domain?	The performance of Qwen2.5 models of different scales on the sentencing prediction task in the legal domain is presented in Table 12.	Generated answer incomplete or uninformative
How does PagedAttention contribute to efficient memory management for large language model serving?	PagedAttention contributes to efficient memory management for large language model serving.	Generated answer incomplete or uninformative

Question Type: Simple
Question: What criteria were used to select papers from PubMed Central (PMC) for inclusion in the Common Pile dataset?
Ground Truth: Papers from PubMed Central (PMC) were selected for inclusion in the Common Pile dataset if their metadata indicated that the publishing journal had designated a CC BY, CC BY-SA, or CC0 license.

Question Type: Reasoning
Question: How might a compromised agent affect a user's digital and economic stability?
Ground Truth: A compromised agent can affect a user's digital and economic stability by inflicting subtle yet significant damage in professional or economic contexts, such as introducing logical errors into computer code, providing flawed data in financial projections, or leaking confidential business strategies. It can also automate disinformation campaigns to manipulate markets. Additionally, it can generate scripts to download malware, trick users into visiting phishing websites, or send phishing emails, thereby damaging the user's reputation and spreading attacks. In severe cases, it can bypass safety protocols to provide harmful instructions, threatening the user's physical safety and financial stability.

Question Type: Multi-Context
Question: How do RL algorithms like PPO, GRPO, and DAPO boost LLMs' reasoning and adaptability?
Ground Truth: RL algorithms like PPO, GRPO, and DAPO boost LLMs' reasoning and adaptability by optimizing the policy model through various techniques. PPO maximizes a clipped surrogate objective to update the policy model using advantage estimation. GRPO estimates the advantage by normalizing group-level rewards, encouraging exploration by removing the KL term. DAPO integrates techniques like a higher clip threshold, dynamic sampling, token-level loss, and overlong reward shaping to ensure a stable and efficient RL process, enhancing the model's reasoning and adaptability.

3.5.3 Evaluation Strategy

Our evaluation procedure was designed to isolate the impact of individual RAG components while ensuring that improvements generalize to unseen questions. First, we evaluated the baseline system using all six selected Ragas metrics, faithfulness, answer relevancy, context precision, context recall, answer semantic similarity, and answer correctness, on the 100 QA pair test set. Next, we focused on optimizing retrieval through improved chunking and embedding models, hybrid search, query optimization, and reranking. In each phase, performance was measured using context precision and context recall, and the configuration with the highest average score across these two metrics was advanced to the next phase while other components remained fixed. This approach isolates the effect of individual system components, ensuring that observed performance changes can be attributed to a single modification, consistent with best-practice recommendations for RAG optimization [102]. After retrieval optimization, we compared different LLMs based on faithfulness and answer relevancy, selecting the model with the highest average score across these two metrics for the final configuration.

All intermediate evaluations were conducted using the 40 QA pair development set. Finally, we assessed the fully optimized pipeline on the 100 QA pair test set using all six metrics, comparing its performance with the baseline to determine whether improvements generalized to unseen data. In all evaluations, we used GPT-4o as the LLM and `text-embedding-3-large` as the embedding model, as described in Section 3.1.

4 Results

This section presents the results of the stepwise optimization process. Starting from the baseline system, we show how successive enhancements to chunking, embedding models, retrieval strategies, and LLMs progressively improved performance, as measured by Ragas metrics. We also compare the baseline and optimized RAG pipelines and assess the robustness of the Ragas evaluations.

4.1 Baseline Performance

We started the evaluation with the baseline approach. Table 8 summarizes the average scores of the baseline RAG pipeline across all evaluation metrics. On average, the system performed strongly in answer relevancy and answer semantic similarity, indicating that the generated answers were usually on-topic and semantically close to the reference answers. Faithfulness and context precision were also relatively high, suggesting that the generated answers generally aligned with the retrieved documents and that the documents contained limited irrelevant content. By contrast, context recall was noticeably lower, implying that the system frequently missed parts of the relevant context. The weakest metric was answer correctness, showing that answers, although often relevant, were not consistently factually accurate.

Table 8: Performance of the baseline RAG pipeline across Ragas metrics.

Metric	Average Score
Faithfulness	0.810
Answer Relevancy	0.887
Context Precision	0.811
Context Recall	0.725
Answer Semantic Similarity	0.875
Answer Correctness	0.495

Figure 11 illustrates the distribution of scores across metrics. Faithfulness, answer relevancy, context precision, and answer semantic similarity are tightly clustered near the upper end of the scale, with narrow interquartile ranges and medians close to the maximum, indicating consistently strong performance across questions. In contrast, answer correctness displays a much wider spread and a notably lower median, suggesting that factual accuracy varies considerably between questions. Context recall also shows greater variability than context precision, with a long lower tail indicating that some questions achieve very low scores. This pattern suggests that while retrieved passages were usually precise, the system sometimes failed to capture all relevant context. Overall, the figure highlights that the baseline system's main limitations lie in incomplete retrieval and factual correctness, rather than in relevance or fluency of the answers. To address these issues, the next stage focuses on optimizing the retrieval components.

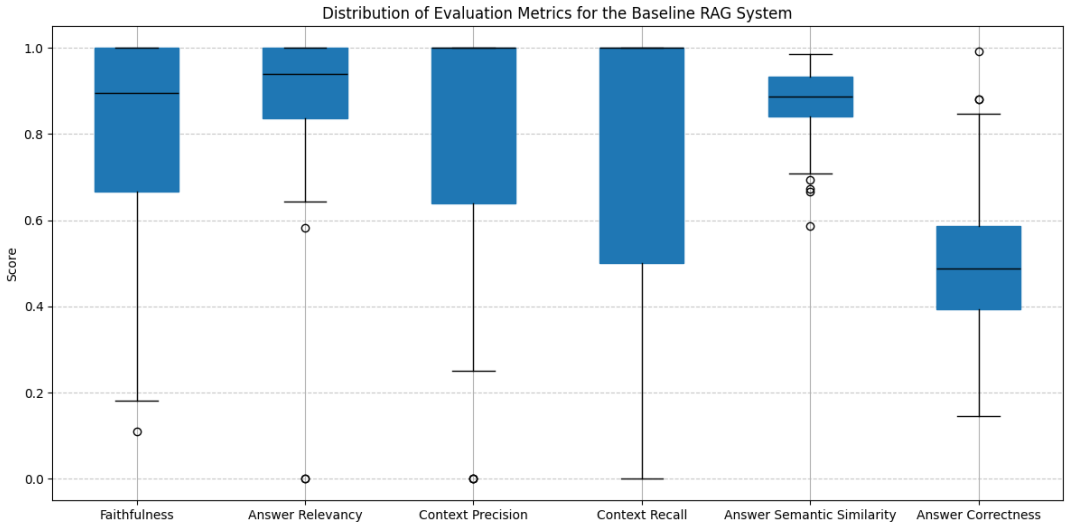


Figure 11: Distribution of Ragas metric scores for the baseline RAG system.

4.2 Retrieval Optimization

This section presents the results of the retrieval component optimization. Starting from the baseline configuration, we examine how incremental modifications to chunking and embedding models, as well as the introduction of hybrid search, query optimization, and reranking, affect retrieval performance. Each stage is evaluated using context precision and context recall to quantify how effectively the system identifies and captures relevant information. The following sections detail the results for each stage and conclude with a summary of cumulative effects.

4.2.1 Chunking Strategies

The retrieval optimization experiments began with an evaluation of different parsing and chunking strategies. The results of these experiments are summarized in Table 9. Performance improved consistently with increasing chunk size across all methods. Configurations with 512 characters achieved average scores in the range of 0.630 to 0.667, while 1024 characters increased this to a range between 0.758 and 0.804. The highest performance was achieved with 2048 characters, yielding averages between 0.839 and 0.870.

The effect of overlap depended on the chunk size. For chunks of 1024 characters, a small overlap led to a slight improvement, whereas for 512 and 2048 characters, it reduced performance. This suggests that adding overlapping text offered little benefit once a single chunk already contained enough contextual information.

Regarding chunking strategies, `RecursiveCharacterTextSplitter` achieved the highest context precision of 0.875 at 2048 characters, Unstructured `hi_res` achieved the highest context recall of 0.874 at 2048 characters, and Unstructured `fast` achieved the highest overall average of 0.870 at 2048 characters. Despite similar performance between the `fast` and `hi_res` strategies, their processing times differed

substantially. Across all configurations, both strategies were run without concurrency: the `fast` variant completed processing in under one minute on average, whereas the `hi_res` strategy consistently required close to an hour. Since the Unstructured `fast` chunking strategy with a chunk size of 2048 had the highest average score, we selected it for our further experiments.

Table 9: Evaluation of different chunking strategies based on context precision and recall. The average column shows the mean of the two metrics. For each configuration, values in parentheses denote the chunk size in characters and the overlap between chunks, expressed as a percentage of the chunk size.

Configuration	Context Precision	Context Recall	Average
RecursiveCharacterTextSplitter (512, 0%)	0.716	0.617	0.667
RecursiveCharacterTextSplitter (512, 10%)	0.748	0.511	0.630
RecursiveCharacterTextSplitter (512, 20%)	0.737	0.574	0.655
RecursiveCharacterTextSplitter (1024, 0%)	0.793	0.722	0.758
RecursiveCharacterTextSplitter (1024, 10%)	0.828	0.698	0.763
RecursiveCharacterTextSplitter (1024, 20%)	0.847	0.745	0.796
RecursiveCharacterTextSplitter (2048, 0%)	**0.875**	0.858	0.867
RecursiveCharacterTextSplitter (2048, 10%)	0.858	0.845	0.852
RecursiveCharacterTextSplitter (2048, 20%)	0.853	0.825	0.839
Unstructured fast (512, 0%)	0.702	0.629	0.666
Unstructured fast (1024, 0%)	0.822	0.757	0.789
Unstructured fast (2048, 0%)	0.871	0.868	**0.870**
Unstructured hi_res (512, 0%)	0.687	0.646	0.666
Unstructured hi_res (1024, 0%)	0.841	0.767	0.804
Unstructured hi_res (2048, 0%)	0.847	**0.874**	0.860

4.2.2 Embedding Models

The next stage of experiments evaluated different embedding models using the optimized chunks. The results are shown in Table 10. The best overall performance was achieved by `text-embedding-3-large`, with the highest context precision of 0.935 and an average score of 0.930. The second-best model was `text-embedding-3-small`, which achieved the highest context recall of 0.927 and an average of 0.912. The older `text-embedding-ada-002` model performed slightly worse, with an average of 0.880.

Among the open-source models, `gte-large-en-v1.5` achieved the strongest results with an average of 0.886, outperforming `text-embedding-ada-002`. The BGE models generally performed less well, with averages ranging between 0.840 and 0.863. Interestingly, the `bge-small-en-v1.5` model achieved higher average

performance than both `bge-base-en-v1.5` and `bge-large-en-v1.5`. Based on these results, we used `text-embedding-3-large` in our further experiments.

Table 10: Evaluation of different embedding models based on context precision and recall. The average column shows the mean of the two metrics.

Model	Context Precision	Context Recall	Average
text-embedding-ada-002	0.912	0.848	0.880
text-embedding-3-small	0.897	**0.927**	0.912
text-embedding-3-large	**0.935**	0.925	**0.930**
bge-small-en-v1.5	0.880	0.846	0.863
bge-base-en-v1.5	0.856	0.825	0.840
bge-large-en-v1.5	0.885	0.799	0.842
gte-base-en-v1.5	0.871	0.881	0.876
gte-large-en-v1.5	0.906	0.866	0.886

4.2.3 Hybrid Search

After the document indexing had been optimized by selecting the best chunking strategy and embedding model, we evaluated how introducing hybrid search affected retrieval performance. The results of the hybrid search experiments with different values of `alpha` are shown in Table 11. An `alpha` of 1, corresponding to pure semantic search, achieved a context precision of 0.937, a context recall of 0.917, and an average score of 0.927. An `alpha` of 0, corresponding to pure keyword search, performed considerably worse, with an average score of 0.856.

Intermediate `alpha` values outperformed both extremes. The best results were obtained with an `alpha` of 0.75, which achieved the highest context precision of 0.960 and the best average score of 0.955. An `alpha` of 0.5, which gives equal weight to both semantic and keyword search, yielded a similar balance between context precision and context recall, with an average score of 0.953. An `alpha` of 0.25, which gives more emphasis to keyword search, performed worse than larger `alpha` values. We continued using an `alpha` of 0.75 in our experiments.

Table 11: Comparison of different `alpha` values in the hybrid retrieval setup based on context precision and recall. The average column shows the mean of the two metrics.

Alpha	Context Precision	Context Recall	Average
1.00	0.937	0.917	0.927
0.75	**0.960**	0.950	**0.955**
0.50	0.953	**0.953**	0.953
0.25	0.942	0.897	0.919
0.00	0.857	0.855	0.856

4.2.4 Query Optimization

To optimize query formulation, two techniques from LangChain were evaluated: `RePhraseQueryRetriever` and `MultiQueryRetriever`. The results are shown in Table 12. Although `MultiQueryRetriever` achieved higher scores than `RePhraseQueryRetriever` in both context precision and context recall, with an average of 0.951, neither method was able to surpass the best result previously achieved through hybrid search alone. Consequently, neither optimization technique was added to the final configuration.

Table 12: Evaluation of different query optimization techniques based on context precision and recall. The average column shows the mean of the two metrics.

Technique	Context Precision	Context Recall	Average
RePhraseQueryRetriever	0.931	0.927	0.929
MultiQueryRetriever	**0.944**	**0.958**	**0.951**

4.2.5 Reranking

In our final retrieval optimization step, we tested different reranker models. The results of our experiments are shown in Table 13. The `bge-reranker-base` achieved an average score of 0.919, which was slightly improved by `bge-reranker-large` at 0.929. ColBERTv2 performed better compared to the BGE rerankers, achieving an average score of 0.951. The strongest performance was obtained with `LLMListwiseRerank`, which achieved the highest context precision of 0.985, the highest context recall of 0.975, and the best overall average score of 0.980. We included the `LLMListwiseRerank` in our final configuration.

Table 13: Evaluation of different reranking models based on context precision and recall. The average column shows the mean of the two metrics.

Model	Context Precision	Context Recall	Average
bge-reranker-base	0.944	0.894	0.919
bge-reranker-large	0.963	0.894	0.929
ColBERTv2	0.971	0.931	0.951
LLMListwiseRerank	**0.985**	**0.975**	**0.980**

4.2.6 Summary of Retrieval Improvements

To provide an overview of the gains achieved during retrieval optimization, we summarize the cumulative impact of each step. Figure 12 illustrates how performance evolved across stages. The baseline approach, evaluated using the same smaller development set of 40 QA pairs applied throughout the retrieval optimization experiments, achieved an average score of 0.776, calculated as the mean of context precision and recall. Optimized chunking increased the average score to 0.870, while replacing the embedding

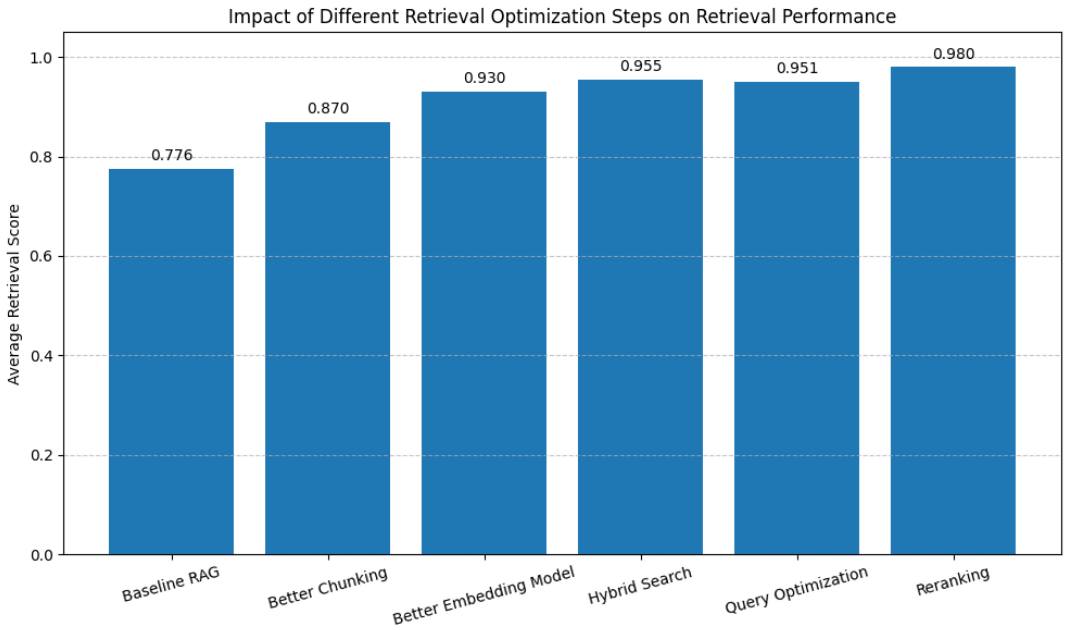


Figure 12: Impact of different retrieval optimization steps on retrieval performance. Each step builds upon the previous one, showing consistent improvements through better chunking, a stronger embedding model, hybrid search, and reranking. Query optimization did not improve results and was excluded from the final configuration.

model further improved the performance to 0.930. Introducing hybrid search raised the score to 0.955. Query optimization did not improve performance and was thus excluded from our final configuration. Finally, reranking improved performance to 0.980.

4.3 Large Language Models

After optimizing the retrieval pipeline, we evaluated how effectively different LLMs utilize the retrieved context to produce coherent responses. The results are shown in Table 14. Among all tested models, GPT-4o achieved the highest overall performance with an average score of 0.913 and was therefore selected for our final RAG configuration. GPT-4o mini followed closely with an average score of 0.909, while GPT-4.1 achieved the highest answer relevancy of 0.919 but a lower faithfulness of 0.864, resulting in a slightly lower average of 0.891. The smaller GPT-4.1 mini performed consistently, while the GPT-4.1 nano showed weaker faithfulness, though it maintained high answer relevancy. The older GPT-3.5 Turbo model was outperformed by the GPT-4o and GPT-4.1 models, but it still achieved a higher average score than GPT-4.1 nano.

Among open-source models, Llama 3.1 8B achieved competitive performance with an average score of 0.875, close to GPT-3.5 Turbo. The smaller Llama 3.2 1B reached an average of 0.816, while Llama 3.2 3B lagged significantly behind at 0.537 due to very low answer relevancy of 0.396. The Gemma 2 models performed moderately,

with the 9B version averaging 0.799 and the 2B version 0.781.

Compared to the ground-truth answers, many models produced more verbose responses, often formatted with bullet points, numbered lists, or concluding remarks. GPT-3.5 Turbo was an exception, as only one of its responses used such formatting. GPT models generally answered more directly, while open-source models frequently prefaced responses with hedging phrases such as "Based on the provided context. . . " or "According to the provided text. . . ". Llama 3.2 3B often responded with statements beginning "I don't know. . . ", and Gemma models occasionally added conversational endings, such as "Let me know if you have any other questions!". Among the open-source models, Llama 3.1 8B produced the most direct answers, stylistically similar to the GPT models.

Table 14: Evaluation of different language models based on faithfulness and answer relevancy. The average column shows the mean of the two metrics.

Model	Faithfulness	Answer Relevancy	Average
GPT-3.5 Turbo	0.863	0.892	0.878
GPT-4o	**0.912**	0.914	**0.913**
GPT-4o mini	0.910	0.907	0.909
GPT-4.1	0.864	**0.919**	0.891
GPT-4.1 mini	0.887	0.901	0.894
GPT-4.1 nano	0.816	0.911	0.864
Llama 3.1 8B	0.874	0.876	0.875
Llama 3.2 3B	0.678	0.396	0.537
Llama 3.2 1B	0.799	0.833	0.816
Gemma 2 9B	0.851	0.747	0.799
Gemma 2 2B	0.788	0.774	0.781

4.4 Optimized RAG Pipeline

Now that retrieval had been optimized and the best LLM had been selected, we evaluated our final RAG configuration using the same six Ragas metrics that we used to evaluate the baseline RAG. The results are presented in Table 15. The system achieved strong performance in retrieval metrics, with context precision of 0.949 and context recall of 0.933. On the generation side, the model scored 0.876 on faithfulness and 0.894 on answer relevancy, indicating that answers were generally grounded in retrieved evidence and aligned with the information need. The answer semantic similarity metric was 0.880, while answer correctness was notably lower at 0.532.

Figure 13 shows the distribution of the optimized RAG scores. Both context precision and context recall are tightly clustered at the top, with very little variation and only a handful of lower values, confirming that the optimized retrieval pipeline consistently returns highly relevant passages. Faithfulness and answer relevancy also remain strong, with most scores in the upper range and a few lower outliers, suggesting that, in some cases, the model still generates answers that are less relevant or not fully

Table 15: Performance of the optimized RAG pipeline across Ragas metrics.

Metric	Average Score
Faithfulness	0.876
Answer Relevancy	0.894
Context Precision	0.949
Context Recall	0.933
Answer Semantic Similarity	0.880
Answer Correctness	0.532

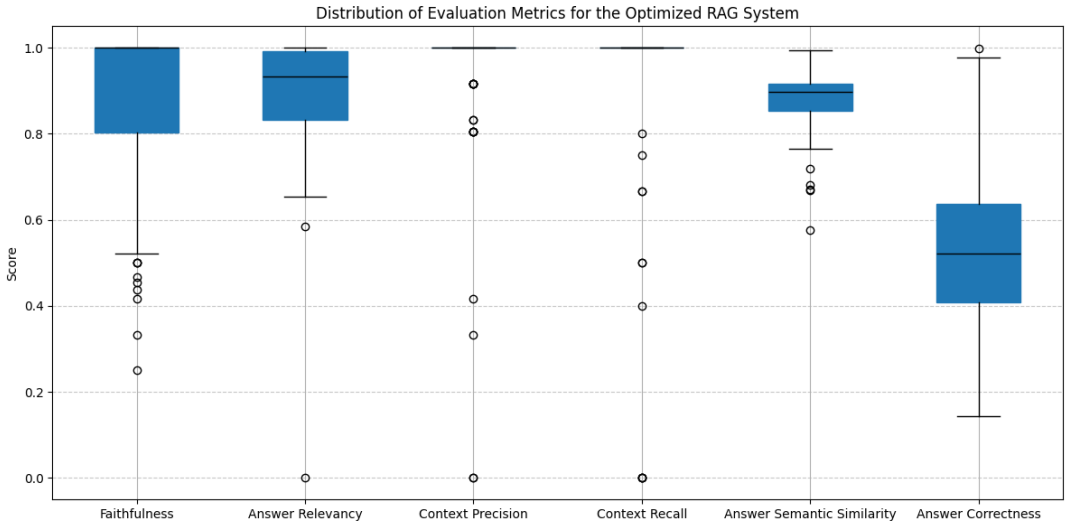


Figure 13: Distribution of Ragas metric scores for the optimized RAG system.

supported by the retrieved context. Answer semantic similarity is similarly stable, with a narrow interquartile range and a high median, indicating that generated responses are usually close in meaning to the reference answers. Answer correctness, while improved compared to the baseline, continues to show the greatest variation, with a broader spread and a noticeably lower median than the other metrics. This confirms that although the optimized configuration produces relevant and well-grounded answers, achieving consistent factual correctness remains the main challenge.

4.5 Comparison of Baseline and Optimized RAG Pipelines

To assess the overall impact of the optimization steps, we compare the optimized pipeline with the baseline. The optimized RAG pipeline demonstrated substantial improvements over the baseline across all evaluation metrics, with particularly strong gains in retrieval quality. As shown in Figure 14, faithfulness increased from 0.810 to 0.876, while context precision and recall rose sharply from 0.811 and 0.725 to 0.949 and 0.933, respectively. In contrast, answer relevancy and semantic similarity remained consistently high, and answer correctness showed a modest improvement, though it continued to be the weakest metric overall. These results indicate that

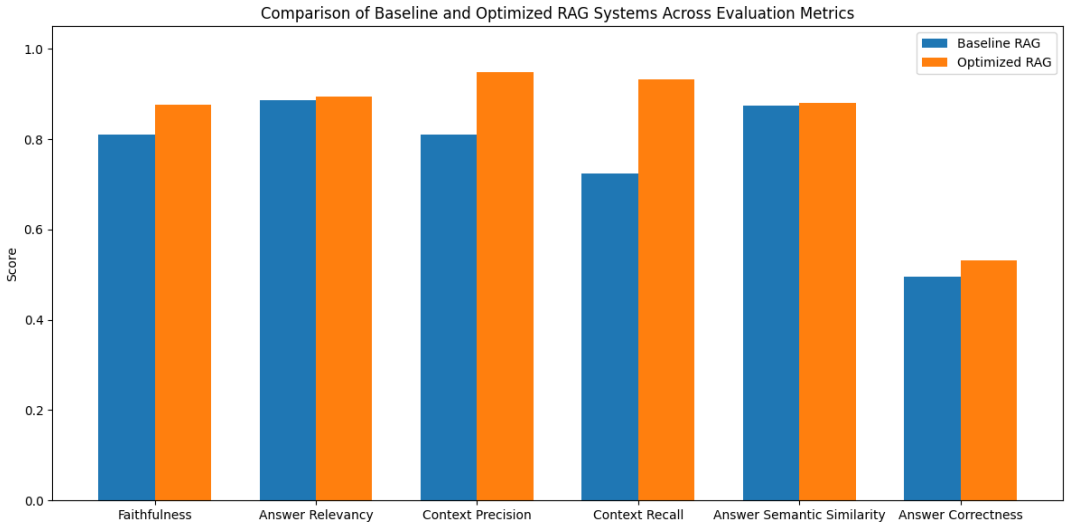


Figure 14: Comparison of baseline and optimized RAG systems across Ragas evaluation metrics. The optimized pipeline shows consistent improvements, particularly in context precision and recall, reflecting enhanced retrieval quality.

retrieval optimization substantially enhanced factual grounding, but factual accuracy remains a key weakness.

To understand how these effects vary by question type, Figure 15 groups the results by simple, reasoning, and multi-context questions. The optimized pipeline yields the largest gains for simple and reasoning questions, especially in faithfulness and retrieval metrics, suggesting that improved retrieval helps the model use context more consistently. However, for multi-context questions, gains in context recall do not translate into higher faithfulness or answer relevancy, highlighting the difficulty of synthesizing information from multiple documents. Answer semantic similarity remains largely unchanged across categories, indicating that the generated responses remain closely aligned with the reference answers in both configurations.

Per-question differences in Figure 16 further illustrate these patterns. Faithfulness shows mostly small improvements with some declines, while answer relevancy remains largely unchanged apart from a few outliers. Context precision improves for many questions, with several remaining unchanged and only a few declines, including one notably larger drop. Context recall shows a broadly positive trend as well, with most questions improving, many remaining unchanged, and only a single clear decline. Answer semantic similarity remains highly stable, with differences clustered close to zero. In contrast, answer correctness displays mixed effects: some questions benefit from optimization, whereas others show slight declines.

To illustrate these patterns qualitatively, Table 16 shows the example for Question 19, which demonstrates a clear improvement: the baseline system failed to retrieve relevant content, whereas the optimized pipeline produced a detailed and factually accurate answer closely aligned with the ground truth. Table 17 shows the example for Question 9, which illustrates a regression, where retrieval failed entirely and led to a nonsensical response. These examples highlight that while optimization improves

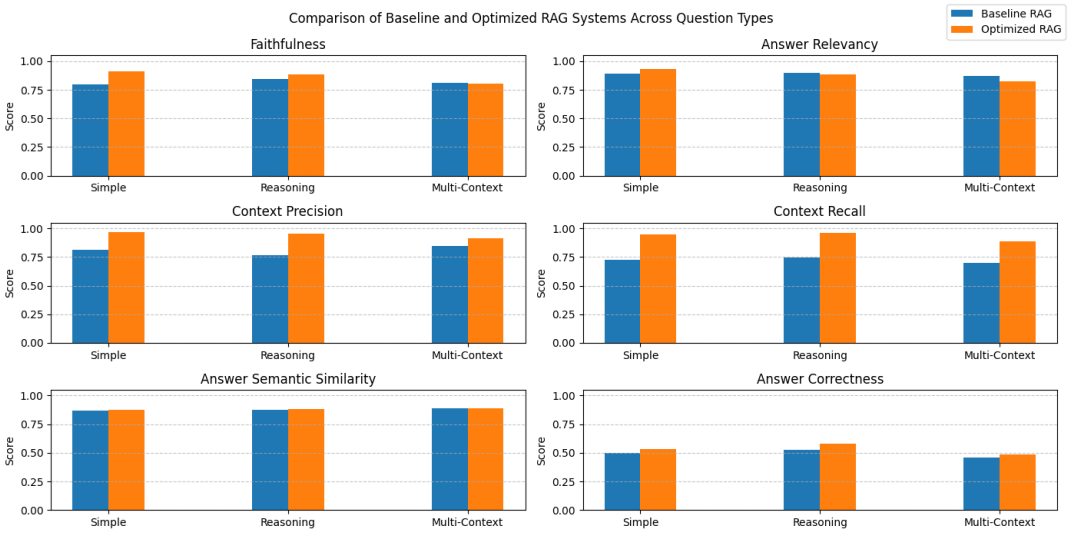


Figure 15: Comparison of baseline and optimized RAG systems across Ragas metrics, grouped by question type. The optimized pipeline shows the largest gains for simple and reasoning questions, particularly in faithfulness and retrieval performance, while improvements for multi-context questions remain limited.

overall performance, it can still occasionally degrade results for specific questions.

4.6 Evaluation Robustness

After optimizing the pipeline, we examined the reliability of the Ragas evaluations. Because LLMs are inherently nondeterministic, we wanted to investigate the degree of variation between different runs. We repeated the evaluation 10 times using fixed generations and contexts produced by the baseline RAG pipeline. To save costs, we used a test set of 20 QA pairs, consisting of 10 simple questions, 5 reasoning questions, and 5 multi-context questions, sampled randomly from the development set of 40 QA pairs. Figure 17 illustrates the distribution of scores across all six metrics.

Although the LLM generations and retrieved contexts were fixed, the results were not identical across runs. This indicates that the LLM evaluator introduces a small degree of nondeterminism, even at temperature 0. The observed variation was minor and did not affect the overall conclusions of this thesis, though it may blur distinctions between approaches with very similar performance.

We also examined the similarity between evaluation results produced by different LLMs. In addition to GPT-4o, the baseline RAG system was evaluated using GPT-4o mini, GPT-4.1, and GPT-4.1 mini on the same 20 QA pair test set used in the previous experiment, with fixed generations and retrievals across all evaluation runs. The results for all six metrics are shown in Figure 18.

The comparison reveals notable variation across LLM evaluators. Faithfulness scores differed considerably, with GPT-4.1 producing the lowest values and GPT-4.1 mini the highest. Answer relevancy also varied, as GPT-4o assigned higher scores than the other models. Context precision was highest for GPT-4o mini, while GPT-4o

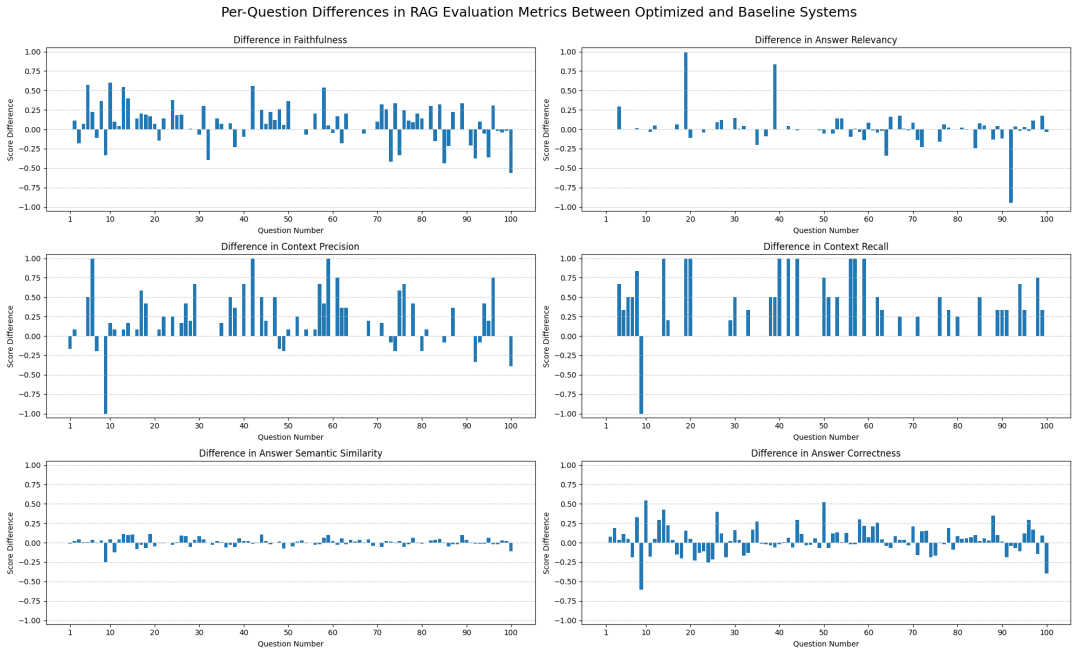


Figure 16: Per-question differences in Ragas metric scores between the baseline and optimized RAG systems. Positive values indicate higher scores for the optimized pipeline, showing consistent gains in retrieval metrics and smaller, mixed effects in generation metrics.

yielded the lowest context recall. Answer semantic similarity remains nearly identical across evaluators, as this metric is embedding-based. In answer correctness, GPT-4o mini produced substantially higher scores than the other models.

Finally, we examined how hallucinated answers were scored. Figure 19 compares the baseline LLM GPT-3.5 Turbo and the strongest LLM GPT-4o, both used without retrieval, against the baseline and optimized RAG pipelines on the 100 QA pair test set, using the answer semantic similarity and answer correctness metrics, which do not depend on the retrieved context but instead compare the generated responses directly to the ground-truth answers.

The results show that hallucinated answers from the no-retrieval baselines achieved substantially lower scores in answer correctness than the RAG pipelines, confirming that retrieval helps reduce factually incorrect responses. At the same time, their semantic similarity scores remained relatively high, indicating that hallucinated answers often resembled ground-truth answers linguistically despite being factually inaccurate. This suggests that answer semantic similarity alone is insufficient to assess answer quality, whereas answer correctness provides a clearer signal of factual grounding. It is also possible that some questions in the test set are general enough to be answered from the parametric knowledge of the LLMs, which explains why the no-retrieval baselines still achieve moderate scores.

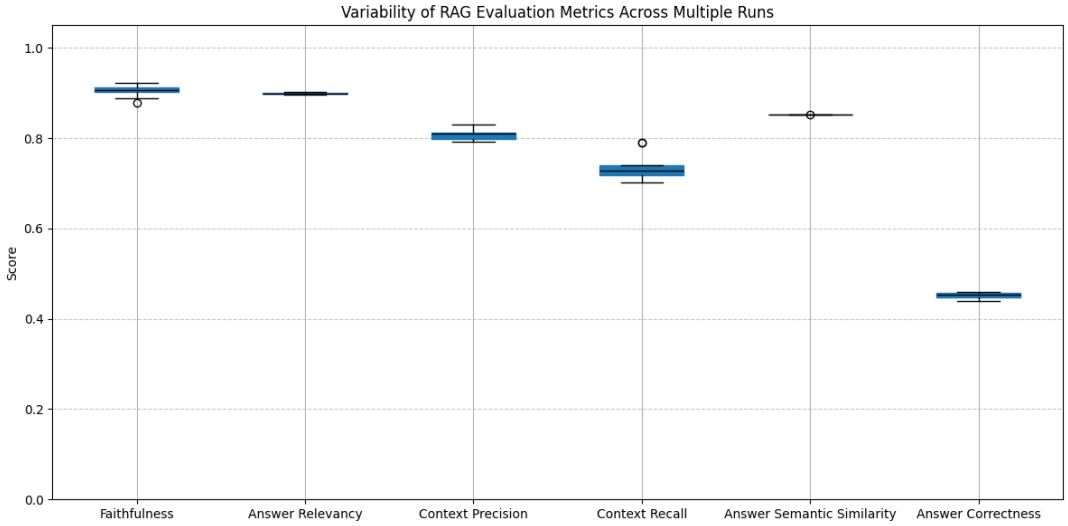


Figure 17: Variability of Ragas evaluation metrics across 10 repeated runs using fixed generations and retrieved contexts from the baseline RAG pipeline. Minor variation across runs indicates limited nondeterminism in the LLM evaluator, even with the temperature set to 0.

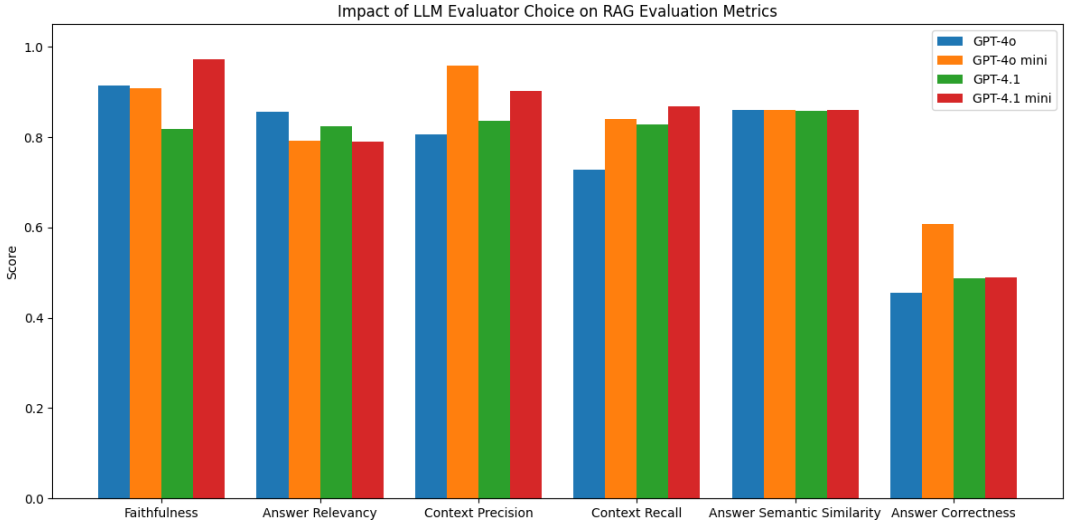


Figure 18: Comparison of Ragas evaluation results obtained using different LLM evaluators on the same fixed baseline RAG outputs. The variation across models highlights that evaluation scores depend on the choice of evaluator.

Table 16: Example comparison for Question 19 showing baseline and optimized RAG responses. The optimized pipeline produces an answer more closely aligned with the ground truth.

Question	How does the performance of smaller models using pure Reinforcement Learning (RL) compare to those distilled from larger reasoning models?
Ground Truth	The performance of smaller models using pure Reinforcement Learning (RL) achieves similar performance on AIME'24 as the distilled version and even outperforms the distilled version on MATH and GPQA, but has slightly lower performance on code benchmarks such as LiveCodeBench.
Baseline RAG Response	The performance of smaller models using pure Reinforcement Learning (RL) can be strong, showcasing generalization ability to out-of-domain tasks. Previous observations suggested that smaller models relying solely on RL may not achieve performance comparable to those distilled from larger reasoning models. However, recent findings contradict this, showing strong results even with pure RL. The specific mechanisms of how RL enhances reasoning capabilities are still debated within the research community.
Optimized RAG Response	The performance of smaller models using pure Reinforcement Learning (RL) is comparable to those distilled from larger reasoning models. Specifically, the Mistral Small 3 model with pure RL achieves similar performance on the AIME'24 benchmark as the distilled version and even outperforms the distilled version on MATH and GPQA benchmarks. However, it has slightly lower performance on code benchmarks such as LiveCodeBench. These findings suggest that the benefits of RL are not exclusive to larger base models and can be effectively applied to smaller models as well. Additionally, applying RL on top of a distilled checkpoint can lead to further performance improvements.

Table 17: Example comparison for Question 9 showing baseline and optimized RAG responses. In this case, the baseline pipeline produces an answer that is more accurate and better aligned with the ground truth.

Question	What is the purpose of implementing a real-time monitoring mechanism in the context of algorithm-generated outputs?
Ground Truth	The purpose of implementing a real-time monitoring mechanism is to track request frequency and resource consumption from individual users or IP addresses, enabling adaptive adjustments to model responses or temporary access restrictions for suspicious users.
Baseline RAG Response	The purpose of implementing a real-time monitoring mechanism in the context of algorithm-generated outputs is to track request frequency, resource consumption, and potentially malicious long outputs in real time during the generation process. It also enables adaptive adjustments to model responses or temporary access restrictions for suspicious users.
Optimized RAG Response	The purpose of implementing a real-time monitoring mechanism in the context of algorithm-generated outputs is to continuously evaluate the execution status of sub-tasks and the completeness of their outcomes. This allows for reflection, re-planning, and re-action based on those outcomes, ensuring that any failures or incomplete results can be addressed dynamically. This mechanism is critical for managing complex planning under uncertainty, preventing non-convergent communication, and minimizing damage from attacks or mistakes by enabling a shorter reaction time. It helps maintain a reliable and secure AI ecosystem by allowing for dynamic updates and adjustments to the execution plan as needed.

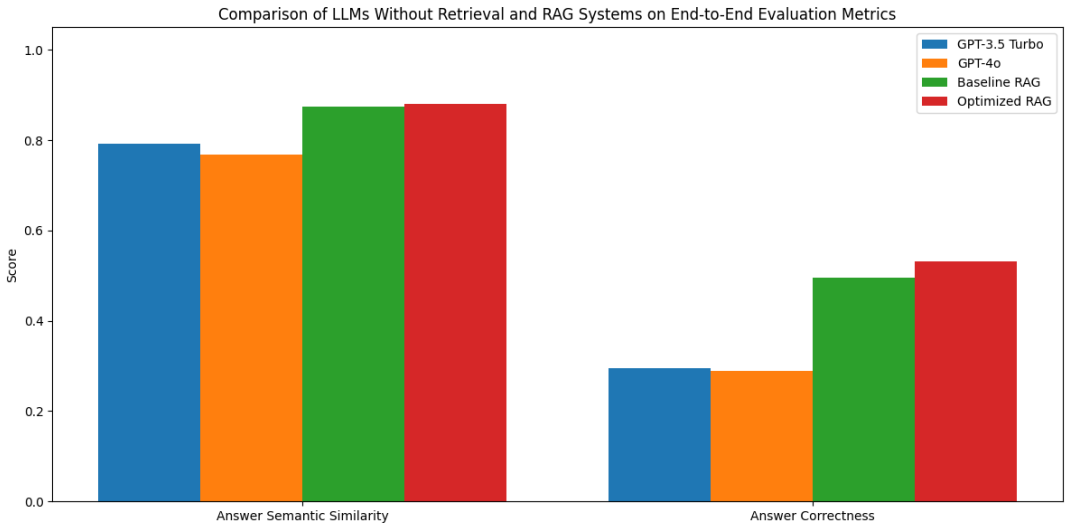


Figure 19: Comparison of LLMs without retrieval and RAG systems on answer semantic similarity and answer correctness. The RAG pipelines achieve substantially higher answer correctness scores, confirming that retrieval reduces hallucinations, while answer semantic similarity remains high across all systems.

5 Discussion

This section interprets the results presented in Section 4, connects them to the research questions introduced in Section 1, and discusses their broader implications. We also address limitations and outline directions for future research.

5.1 Optimizing Retrieval in RAG Pipelines

Our first research question asked how the retrieval process in a RAG pipeline can be improved through component-level optimization. Starting from a baseline system, we sequentially tested document parsing and chunking techniques, embedding models, hybrid search, query optimization, and reranking. The optimized pipeline significantly improved retrieval performance in our test set, with context precision increasing from 0.811 to 0.949 and context recall from 0.725 to 0.933. These gains illustrate that substantial improvements in retrieval performance can be achieved through careful selection and tuning of pipeline components. Next, we analyze the impact of each optimization step in more detail.

Chunking The size of the chunks had the greatest impact on retrieval. Across both `RecursiveCharacterTextSplitter` and Unstructured variants, chunks of around 2048 characters consistently yielded higher context precision and context recall values than smaller chunks, while overlap added no benefit and sometimes reduced performance. The strong performance of larger chunks may be partly due to the way the QA test set was constructed. Since the questions and answers were generated from chunks of roughly 4096 characters, smaller chunks may not have included enough relevant information to align with the ground-truth answers, putting them at a disadvantage. This indicates that evaluation design can shape which chunking strategies seem most effective, so the results should be interpreted with some caution. At the same time, the advantage of larger chunks is unlikely to come only from this bias. Scientific papers often contain long, coherent passages, and larger chunks are better at keeping these sections intact. The lack of a clear advantage for the Unstructured `hi_res` strategy likely stems from the nature of our QA test set, which consisted mainly of text-based questions that did not benefit from the enhanced parsing capabilities of the `hi_res` approach. As a result, the additional parsing granularity it provides offered little benefit over the `fast` variant, and neither Unstructured approach substantially outperformed the simpler `RecursiveCharacterTextSplitter`.

Findings from prior studies are mixed. Yepes et al. [90] compared token-based chunking at 128, 256, and 512 tokens with an Unstructured method similar to our `hi_res` 2048 character approach, using financial documents. They found that the Unstructured strategy outperformed token-based chunking across both retrieval and question-answering accuracy. Wang et al. [95] found that intermediate sizes from 256 to 512 tokens performed best, while larger chunks of 2048 tokens performed noticeably worse. Smith et al. [88] reported that `RecursiveCharacterTextSplitter` was consistently strong for smaller chunks, outperforming token-based chunking at 400 tokens or less without overlap, although this advantage did not hold for larger chunks

with overlap. Finally, Li et al. [103] compared smaller chunk sizes from 48 to 192 tokens and found only minimal performance differences, with the largest chunks performing slightly better on some metrics. Together, these results suggest that the most effective chunking strategy depends heavily on both the corpus and the evaluation setup. In our case, the structure of scientific papers appears to favor larger chunks, whereas in domains with shorter passages or noisier queries, smaller chunks may be more effective.

Hybrid Search To find the balance between semantic and keyword search, we tested several `alpha` values in hybrid retrieval. The best results were achieved with `alpha` 0.75, followed closely by 0.50, indicating that including keyword search improves retrieval when semantic search alone misses acronyms or very specific terms. In contrast, `alpha` 0.25 was weaker, and BM25 alone performed clearly worse than semantic search. These findings are consistent with those of Wang et al. [95], who also found that hybrid search outperforms semantic search alone.

Query Optimization As a next step, we examined query optimization using two LangChain techniques: `RePhraseQueryRetriever` and `MultiQueryRetriever`. While `MultiQueryRetriever` performed slightly better, likely because it retrieves more documents, both approaches lowered performance compared to using the original queries. The questions in our test set contained few unnecessary details or unclear phrasing, so the transformed queries were often very similar to the original ones, as shown in Table 3. We used a temperature of 0 for the LLM that generated alternative queries. A higher temperature or different prompts might have produced more diverse queries, but we did not explore this further. In our setting, query optimization did not provide a benefit and was therefore omitted from the final configuration.

Li et al. [103] evaluated query expansion that generates multiple alternatives for the original query and did not observe meaningful performance gains. Wang et al. [95] compared several techniques: query rewriting, decomposition into sub-questions, and HyDE (Hypothetical Document Embeddings) [104]. They found that rewriting and decomposition did not improve performance, while HyDE did. Because our queries were already concise and specific, this likely explains why query rewriting or expansion did not help in our experiments.

Reranking We explored reranking in the post-retrieval phase and found mixed outcomes. `LLMListwiseRerank` clearly performed best, improving both context precision and context recall, while all other rerankers lowered performance compared to not using reranking at all. This suggests that reranking is not inherently beneficial and its effectiveness depends strongly on the approach used. In particular, listwise reranking, which considers the relative quality of multiple candidates, appears more effective than pointwise methods. Wang et al. [95] also emphasized the role of reranking, reporting that omitting it led to a noticeable drop in performance, even though they tested different rerankers than us.

Key Takeaways Our experiments show that retrieval performance in RAG can be improved substantially through careful component-level optimization. The largest gains came from better parsing and chunking, where larger chunks of around 2048 characters consistently outperformed smaller ones, while overlap provided no benefit. We also observed substantial improvements from stronger embedding models, which are discussed further in Section 5.2. Hybrid search proved useful for capturing acronyms and specific terms, while query optimization lowered performance in our setting, likely because the queries were already concise. Reranking was only beneficial with the LLM-based listwise approach, suggesting that reranking effectiveness depends strongly on the method used.

Although our study focused on scientific papers in PDF format, the retrieval improvements we observed are likely transferable to other English PDF documents, as none of the evaluated approaches were specifically optimized for this domain. Parsing scientific papers is particularly challenging compared to documents in domains such as finance or law [105], which suggests that our pipeline may perform even better in those settings. However, confirming this remains a question for future work.

5.2 Comparing Open-Source and Proprietary Models in RAG

The second research question investigated how open-source embedding models and LLMs compare to proprietary ones and how well leaderboards predict their downstream performance.

Embedding Models Among embeddings, proprietary OpenAI models performed best overall. The `text-embedding-3-large` achieved an average score of 0.930, the highest among all models tested, with `text-embedding-3-small` close behind at 0.912. Both outperformed the older `text-embedding-ada-002`, which achieved an average score of 0.880. In our evaluation, this shows a clear improvement from `text-embedding-ada-002` to the newer `text-embedding-3-small` and `text-embedding-3-large` models, with the large variant performing best.

Open-source models showed more mixed results. The strongest was `gte-large-en-v1.5`, which achieved an average score of 0.886, slightly higher than `text-embedding-ada-002`. The BGE models exhibited surprising behavior: the small variant outperformed the large one, with average scores of 0.863 and 0.842, respectively, indicating that larger parameter counts do not necessarily translate into improved retrieval performance. Compared to GTE, BGE models have smaller input limits, as shown in Table 1. This did not affect our experiments, but it does limit their use when larger chunks are needed. Contrary to our findings, Wang et al. [95] reported that BGE models outperformed `gte-large-en-v1.5` on a different dataset, underscoring how evaluation outcomes depend heavily on the corpus and setup.

The leaderboard results only partially matched our findings. On MTEB, GTE models ranked highest among open-source options, while `text-embedding-ada-002` ranked lowest. However, in our experiments, `text-embedding-ada-002` performed much better than expected, surpassing all BGE models and `gte-base-en-v1.5`,

showing that a lower MTEB score does not necessarily predict weaker retrieval performance. On MTEB, average scores for the selected models are consistently higher than retrieval-specific scores, as shown in Table 2, reflecting that the selected models are general-purpose and tend to perform better on tasks other than retrieval. This shows why it is important to look beyond the overall averages when selecting models for RAG. GTE models performed strongly both on MTEB and in our experiments, but the discrepancy for `text-embedding-ada-002` indicates that leaderboard rankings provide only a rough indication of quality and cannot reliably predict retrieval effectiveness across domains.

Large Language Models Among proprietary models, GPT-4o achieved the highest performance with an average score of 0.913, followed closely by GPT-4o mini at 0.909. Both outperformed the newer GPT-4.1 models, which achieved average scores of 0.891 for GPT-4.1 and 0.894 for GPT-4.1 mini. Even the older GPT-3.5 Turbo, with a score of 0.878, surpassed GPT-4.1 nano, which had an average score of 0.864. Although GPT-4.1 has been shown to handle longer contexts more effectively [106], this advantage did not translate into improved performance in our RAG experiments. The retrieved passages were relatively short, limiting the model's potential benefit from its expanded context window.

Open-source LLMs performed worse than proprietary models but showed competitive results in some cases. Llama 3.1 8B reached an average score of 0.875, nearly matching GPT-3.5 Turbo and outperforming GPT-4.1 nano. The smaller Llama 3.2 1B also performed respectably at 0.816, surpassing Gemma 2 9B at 0.799. By contrast, Llama 3.2 3B performed markedly worse than all other models, with an average score of 0.537. These surprising findings highlight that model size alone is not a reliable predictor of performance in RAG. Li et al. [103] reached similar conclusions when comparing different Mistral variants, finding that larger models did not always produce better results.

The differences we observed in response style in Section 4.3 help explain these outcomes. GPT models tended to provide direct answers, whereas many open-source models expressed uncertainty in their responses. The poor performance of Llama 3.2 3B can largely be attributed to its frequent use of "I don't know" statements, which prevented it from producing clear answers. Gemma models often added conversational endings, suggesting they were tuned more for general chat than for fact-based QA. This helps explain why the GPT models and Llama 3.1 8B performed strongly in our evaluation. These patterns suggest that, in addition to the content of the generated responses, the response style also plays a significant role in the effectiveness of RAG.

On the leaderboards, the top proprietary and open-source LLMs were GPT-4.1 and Gemma 2 9B, respectively, whereas in our evaluation, GPT-4o and Llama 3.1 8B achieved the highest scores. Llama 3.2 3B was also substantially weaker in our tests than its leaderboard position suggested. These discrepancies indicate that leaderboard positions do not reliably predict performance in retrieval-based question answering. Therefore, models should be validated directly in the target domain rather than relying on leaderboard scores.

Key Takeaways Our experiments demonstrate that proprietary models achieved the strongest performance in RAG, though some open-source models were competitive. Among embeddings, OpenAI's `text-embedding-3-large` delivered the highest scores, while `gte-large-en-v1.5` was strong enough to rival the older `text-embedding-ada-002`. For LLMs, GPT-4o achieved the best overall results, but Llama 3.1 8B performed comparably to GPT-3.5 Turbo, and even the smaller Llama 3.2 1B surpassed larger models such as Gemma 2 9B, confirming that model size alone is not a reliable predictor of effectiveness. Leaderboard rankings captured broad trends but often diverged from our results, reinforcing the need to validate models in the target domain rather than relying solely on external benchmarks.

5.3 Evaluating RAG with LLM-as-a-Judge

Our third research question examined how RAG can be evaluated with LLM-as-a-judge, leveraging the test set generation and evaluation capabilities of the Ragas library.

We created a QA test set using the synthetic generation capabilities of Ragas and manually reviewed the outputs. Some questions lacked proper ground-truth answers, and some generated answers only partially addressed the questions, as noted in Section 3.5.2. This highlights the need for human review before using such test sets in evaluation.

In the evaluation phase, Ragas was used for both component-wise and end-to-end assessments. Context precision and context recall clearly reflected differences across configurations, showing their usefulness for measuring retrieval quality. Faithfulness and answer relevancy also varied across LLMs, indicating sensitivity to how models use retrieved context. In contrast, end-to-end metrics were less effective at distinguishing responses from the optimized RAG pipeline and the baseline: answer semantic similarity remained high for both configurations, and answer correctness improved only marginally despite notable gains in component-wise metrics, consistently yielding lower scores than the other metrics. Among the different question types, multi-context questions proved especially challenging, showing only minor gains from the optimized pipeline compared to simpler ones.

To assess robustness, we repeated evaluations using the same answers and contexts and observed some variance due to evaluator randomness. Using different LLM evaluators introduced additional differences across several metrics. These findings highlight that scores from one LLM evaluator are not necessarily comparable to those from another, and combining multiple evaluators may be more reliable despite higher costs. Our comparison was restricted to the GPT-4o and GPT-4.1 variants. Extending the evaluation to include LLMs from other providers would likely reveal even larger variations in scoring.

A key limitation of answer semantic similarity as a metric was its inability to distinguish hallucinated from grounded answers. Responses could receive high scores even when factually wrong if they stayed on topic, showing that semantic similarity is not a reliable indicator of factual grounding. Answer correctness also faced systematic issues. Verbose model outputs often led Ragas to extract false positives, since they contained additional statements not present in the ground-truth answers. At the same

time, false negatives occurred when the generated answers omitted details present in the ground-truth answers. Prompting LLMs to produce more concise outputs might reduce false positives, but since the ground-truth answers were LLM-generated rather than expert-reviewed, the overall scores should be interpreted with some caution, since we do not know how detailed answers real users would prefer.

Roychowdhury et al. [107] reached similar conclusions. They found faithfulness and the factual correctness part of answer correctness to be useful indicators, but questioned the reliability of answer relevancy and semantic similarity. They also criticized Ragas for working like a black box, a limitation we encountered as well. To address this, we used LangSmith to inspect intermediate LLM outputs. In practice, LLM observability tools such as LangSmith could also be used to monitor user queries and refine test sets based on actual usage, making them a valuable complement to Ragas.

Overall, Ragas offers a practical starting point for evaluating RAG through synthetic test generation and automated metrics. Manual review of the generated test set remains necessary to ensure quality, ideally with domain experts refining the questions and answers. While considerably cheaper and faster than large-scale human evaluation, Ragas-based assessment is not cost-free: evaluating multiple RAG configurations can still cost tens of dollars even for small test sets, due to the multiple few-shot prompts used internally by the framework. Careful test design and the selective use of metrics are therefore important to keep evaluation costs manageable. As such, LLM-as-a-judge evaluations should serve as a complement rather than a substitute for human assessment. Ultimately, real-world systems must be tested at runtime against unpredictable user queries, where evaluation frameworks alone cannot anticipate all failure cases [108].

5.4 Limitations

Despite promising results, our study has several limitations that should be acknowledged. While this thesis demonstrates that LLM-as-a-judge can be a practical approach for evaluating RAG systems, it also comes with important caveats. GPT-4o was both the primary evaluator and the strongest model in our experiments, raising the possibility of self-enhancement bias [11]. We also observed variation across repeated runs and between evaluator models, indicating that evaluation outcomes are not fully stable. These factors mean that the results should be interpreted with some caution. Furthermore, we did not include human evaluation or involve domain experts in reviewing the generated QA test sets, which limits our confidence in how well the results would transfer to real usage.

The scope of our evaluation data was also limited. Due to the cost of Ragas and the long processing times of the Unstructured `hi_res` partitioning strategy, we relied on a relatively small test set and a limited number of papers. Our parsing pipeline did not capture figures and only partially captured tables and equations, so the QA test sets lacked questions about such content. If the test data had included these elements or broader queries requiring synthesis across multiple papers, our RAG pipeline would likely have performed worse. More generally, focusing only on scientific papers limits

the generalizability of our findings to other domains.

Our methodology also had constraints. We optimized the RAG pipeline component by component using a greedy strategy, which risks overlooking combinations that could perform better together. Averaging metrics into a single score further simplifies trade-offs and can obscure weaknesses in specific dimensions. In addition, we focused purely on retrieval and generation performance, without considering speed or cost. This risks favoring slower or more expensive configurations for only marginal performance improvements. We also did not evaluate aspects such as noise robustness, negative rejection, information integration, or counterfactual robustness, all of which are considered essential abilities of RAG, as we learned in Section 2.5.2.

Finally, our experiments were limited by available resources. Hardware constraints prevented us from testing larger embedding models, rerankers, or LLMs. We also did not explore prompt engineering or tuning parameters, such as temperature or the number of retrieved documents, to keep the number of experiments feasible. Given the rapid pace of progress in RAG, it was not possible to cover all new models and methods within the scope of this thesis.

5.5 Future Work

This thesis lays the foundation for several future research directions. A first priority is improving document processing. Our parsing strategies, based on pypdf and Unstructured, struggled with figures, tables, and equations, which are central to research papers. Specialized tools for scientific documents, such as GROBID [109] and Nougat [110], could be evaluated as alternatives. Another direction is vision language models (VLMs), which are LLMs that can process images in addition to text. General-purpose LLMs such as GPT-4o can process both text and images, and have already shown stronger parsing performance than Unstructured [111]. An especially interesting direction is ColPali [112], which combines a VLM with a ColBERT-style late interaction mechanism to produce embeddings directly from page images, thereby eliminating the need for separate parsing or chunking.

Retrieval itself could also be enhanced. One direction is metadata enrichment, in which chunks are annotated with attributes such as author, publication date, or relevant keywords, enabling more precise filtering of results. Prior work has shown that metadata can improve retrieval effectiveness [108]. Another promising technique is HyDE [104], which generates a synthetic document to represent the query. By aligning query representations more closely with the style and semantics of corpus documents, HyDE has improved retrieval in earlier studies [95] and could be effective in this setting as well.

Future work could also explore improvements to the models. Since no models were trained during this thesis, one extension would be to fine-tune an embedding model or a reranker on research paper data to better capture domain-specific terminology and potentially improve accuracy. Evaluating a wider range of embeddings, rerankers, and LLMs, including those from organizations not covered in this thesis, would also strengthen the conclusions. In addition, prompt engineering remains underexplored: even small changes to prompts can affect performance [103], making this a low-cost

but potentially high-impact area for optimization.

For real-world use, developing the system into a chat application would introduce additional requirements. Conversational memory would be needed to retain context across turns, and answers should include source citations for transparency. The system would also need to scale to a much larger corpus and support continuous updates, for instance, by periodically ingesting newly published papers. A further step is to integrate multiple sources beyond a static knowledge base, combined with external tools such as web search and APIs for services like arXiv. Moving in this direction points toward modular architectures for research assistants, an idea explored in earlier work such as [113, 114, 115, 116].

Finally, evaluation methods offer another direction for extension. Although Ragas provided a convenient framework, comparing its results with human judgments by domain experts and with alternative LLM-as-a-judge approaches would give a more complete picture of its usefulness and reliability. A more advanced parsing approach could also yield richer test sets containing questions about figures or tables, enabling finer distinctions between methods. Robustness testing, including noisy queries, adversarial inputs, and out-of-domain questions, would further clarify the system's limits.

6 Conclusion

This thesis examined how retrieval-augmented generation (RAG) can be systematically improved and evaluated. It focused on optimizing retrieval through component-level analysis, comparing open-source and proprietary models, and exploring the use of LLM-as-a-judge for automated evaluation.

We began with a naive baseline system using simple vector search, unoptimized chunks, and an older embedding model and LLM. Through iterative evaluation with Ragas, we optimized the pipeline step by step: improving parsing and chunking, testing embedding models, introducing hybrid search, trying query optimization, and applying reranking. With the optimized retrieval pipeline, we then compared a range of LLMs.

Our results show that retrieval performance improves substantially with careful optimization. Larger chunks, around 2048 characters, consistently outperformed smaller ones. Hybrid search proved superior to pure semantic or keyword methods, and reranking only helped with the LLM-based listwise approach. By contrast, query optimization did not lead to consistent improvements. Among embedding models, OpenAI's proprietary variants performed best overall, though `gte-large-en-v1.5` remained competitive. For LLMs, GPT-4o achieved the strongest results, while Llama 3.1 8B performed surprisingly well. In contrast, Llama 3.2 3B underperformed, suggesting that model size and leaderboard rankings are unreliable predictors of effectiveness in RAG. Ragas evaluations also revealed differences across metrics: context precision, context recall, and faithfulness proved useful, whereas answer semantic similarity failed to detect hallucinations and remained largely unchanged even as response quality improved.

Together, the results underscore both the potential and the limitations of current RAG systems. Significant gains can be achieved through systematic optimization, but success ultimately depends on domain-specific validation rather than assumptions about model size or leaderboard position. Although proprietary models delivered the best results overall, the narrowing gap with open-source models indicates that high performance is no longer limited to proprietary solutions.

Beyond research, these findings have practical relevance. Organizations adopting RAG should validate models in their own settings, combine automated evaluation with human oversight, and continuously monitor usage and refine the system to ensure reliability.

While the study was constrained by limited computational resources, a small test set, and the high cost of large-scale evaluations, the findings still highlight clear directions for improvement. Future work should focus on enhancing the parsing of complex documents, developing richer retrieval methods such as filtering results based on metadata, and improving evaluation by combining automated metrics with expert review. Ultimately, practical RAG systems will require not only optimized pipelines but also continuous monitoring and adaptation to real user queries.

References

[1] OpenAI, "Introducing ChatGPT," 2022, Accessed: Sep. 30, 2025. [Online]. Available: https://openai.com/index/chatgpt/

[2] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Li, X. Tang, Z. Liu, P. Liu, J.-Y. Nie, and J.-R. Wen, "A Survey of Large Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2303.18223

[3] N. Kandpal, H. Deng, A. Roberts, E. Wallace, and C. Raffel, "Large Language Models Struggle to Learn Long-Tail Knowledge," in *Proceedings of the 40th International Conference on Machine Learning*, ser. ICML'23. JMLR.org, 2023. [Online]. Available: https://proceedings.mlr.press/v202/kandpal23a.html

[4] Y. Zhang, Y. Li, L. Cui, D. Cai, L. Liu, T. Fu, X. Huang, E. Zhao, Y. Zhang, Y. Chen, L. Wang, A. T. Luu, W. Bi, F. Shi, and S. Shi, "Siren's Song in the AI Ocean: A Survey on Hallucination in Large Language Models," *Computational Linguistics*, pp. 1–46, 09 2025. [Online]. Available: https://doi.org/10.1162/COLI.a.16

[5] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel *et al.*, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/6b493230205f780e1bc26945df7481e5-Paper.pdf

[6] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-Augmented Generation for Large Language Models: A Survey," 2024. [Online]. Available: https://arxiv.org/abs/2312.10997

[7] N. Muennighoff, N. Tazi, L. Magne, and N. Reimers, "MTEB: Massive Text Embedding Benchmark," in *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*, A. Vlachos and I. Augenstein, Eds. Dubrovnik, Croatia: Association for Computational Linguistics, May 2023, pp. 2014–2037. [Online]. Available: https://aclanthology.org/2023.eacl-main.148

[8] C. Fourrier, N. Habib, A. Lozovskaya, K. Szafer, and T. Wolf, "Open LLM Leaderboard v2," https://huggingface.co/spaces/open-llm-leaderboard/open_llm_leaderboard, 2024, Accessed: Sep. 2, 2025.

[9] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, B. Zhu, H. Zhang, M. Jordan, J. E. Gonzalez, and I. Stoica, "Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference," in *Proceedings of the 41st International Conference on Machine Learning*, ser.

Proceedings of Machine Learning Research, R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp, Eds., vol. 235. PMLR, 21–27 Jul 2024, pp. 8359–8388. [Online]. Available: https://proceedings.mlr.press/v235/chiang24b.html

[10] A. B. Sai, A. K. Mohankumar, and M. M. Khapra, "A Survey of Evaluation Metrics Used for NLG Systems," *ACM Comput. Surv.*, vol. 55, no. 2, Jan. 2022. [Online]. Available: https://doi.org/10.1145/3485766

[11] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena," in *Thirty-seventh Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, 2023. [Online]. Available: https://openreview.net/forum?id=uccHPGDlao

[12] S. Es, J. James, L. Espinosa Anke, and S. Schockaert, "RAGAs: Automated Evaluation of Retrieval Augmented Generation," in *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*, N. Aletras and O. De Clercq, Eds. St. Julians, Malta: Association for Computational Linguistics, Mar. 2024, pp. 150–158. [Online]. Available: https://aclanthology.org/2024.eacl-demo.16

[13] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, J. Burstein, C. Doran, and T. Solorio, Eds. Minneapolis, Minnesota: Association for Computational Linguistics, Jun. 2019, pp. 4171–4186. [Online]. Available: https://aclanthology.org/N19-1423

[14] A. Radford, K. Narasimhan, T. Salimans, and I. Sutskever, "Improving Language Understanding by Generative Pre-Training," 2018. [Online]. Available: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf

[15] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," 2019. [Online]. Available: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf

[16] P. Gage, "A New Algorithm for Data Compression," *C Users J.*, vol. 12, no. 2, p. 23–38, feb 1994. [Online]. Available: https://dl.acm.org/doi/10.5555/177910.177914

[17] R. Sennrich, B. Haddow, and A. Birch, "Neural Machine Translation of Rare Words with Subword Units," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long*

Papers), K. Erk and N. A. Smith, Eds. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1715–1725. [Online]. Available: https://aclanthology.org/P16-1162

[18] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," in *Advances in Neural Information Processing Systems*, H. Larochelle, M. Ranzato, R. Hadsell, M. Balcan, and H. Lin, Eds., vol. 33. Curran Associates, Inc., 2020, pp. 1877–1901. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2020/file/1457c0d6bfcb4967418bfb8ac142f64a-Paper.pdf

[19] Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, J. Klingner, A. Shah, M. Johnson, X. Liu, Łukasz Kaiser, S. Gouws, Y. Kato, T. Kudo, H. Kazawa, K. Stevens, G. Kurian, N. Patil, W. Wang, C. Young, J. Smith, J. Riesa, A. Rudnick, O. Vinyals, G. Corrado, M. Hughes, and J. Dean, "Google's Neural Machine Translation System: Bridging the Gap between Human and Machine Translation," 2016. [Online]. Available: https://arxiv.org/abs/1609.08144

[20] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," *Advances in Neural Information Processing Systems*, vol. 30, 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf

[21] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, p. 1735–1780, Nov. 1997. [Online]. Available: https://doi.org/10.1162/neco.1997.9.8.1735

[22] D. Bahdanau, K. Cho, and Y. Bengio, "Neural Machine Translation by Jointly Learning to Align and Translate," 2016. [Online]. Available: https://arxiv.org/abs/1409.0473

[23] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 7871–7880. [Online]. Available: https://aclanthology.org/2020.acl-main.703

[24] Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov, "RoBERTa: A Robustly

Optimized BERT Pretraining Approach," 2019. [Online]. Available: https://arxiv.org/abs/1907.11692

[25] OpenAI, "GPT-4 Technical Report," 2024. [Online]. Available: https://arxiv.org/abs/2303.08774

[26] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, A. Rodriguez, A. Joulin, E. Grave, and G. Lample, "LLaMA: Open and Efficient Foundation Language Models," 2023. [Online]. Available: https://arxiv.org/abs/2302.13971

[27] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom, "Llama 2: Open Foundation and Fine-Tuned Chat Models," 2023. [Online]. Available: https://arxiv.org/abs/2307.09288

[28] Llama Team, AI@Meta, "The Llama 3 Herd of Models," 2024. [Online]. Available: https://arxiv.org/abs/2407.21783

[29] Gemma Team, Google DeepMind, "Gemma: Open Models Based on Gemini Research and Technology," 2024. [Online]. Available: https://arxiv.org/abs/2403.08295

[30] ——, "Gemma 2: Improving Open Language Models at a Practical Size," 2024. [Online]. Available: https://arxiv.org/abs/2408.00118

[31] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. F. Christiano, J. Leike, and R. Lowe, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 27 730–27 744. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/b1efde53be364a73914f58805a001731-Paper-Conference.pdf

[32] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei, "Deep Reinforcement Learning from Human Preferences," in *Advances in Neural Information Processing Systems*, I. Guyon, U. V. Luxburg, S. Bengio,

H. Wallach, R. Fergus, S. Vishwanathan, and R. Garnett, Eds., vol. 30. Curran Associates, Inc., 2017. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2017/file/d5e2c0adad503c91f91df240d0cd4e49-Paper.pdf

[33] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," 2017. [Online]. Available: https://arxiv.org/abs/1707.06347

[34] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," 2020. [Online]. Available: https://arxiv.org/abs/2001.08361

[35] J. Hoffmann, S. Borgeaud, A. Mensch, E. Buchatskaya, T. Cai, E. Rutherford, D. de las Casas, L. A. Hendricks, J. Welbl, A. Clark, T. Hennigan, E. Noland, K. Millican, G. van den Driessche, B. Damoc, A. Guy, S. Osindero, K. Simonyan, E. Elsen, O. Vinyals, J. W. Rae, and L. Sifre, "An empirical analysis of compute-optimal large language model training," in *Advances in Neural Information Processing Systems*, A. H. Oh, A. Agarwal, D. Belgrave, and K. Cho, Eds., 2022. [Online]. Available: https://openreview.net/forum?id=iBBcRUlOAPR

[36] I. R. McKenzie, A. Lyzhov, M. M. Pieler, A. Parrish, A. Mueller, A. Prabhu, E. McLean, X. Shen, J. Cavanagh, A. G. Gritsevskiy, D. Kauffman, A. T. Kirtland, Z. Zhou, Y. Zhang, S. Huang, D. Wurgaft, M. Weiss, A. Ross, G. Recchia, A. Liu, J. Liu, T. Tseng, T. Korbak, N. Kim, S. R. Bowman, and E. Perez, "Inverse Scaling: When Bigger Isn't Better," *Transactions on Machine Learning Research*, 2023, featured Certification. [Online]. Available: https://openreview.net/forum?id=DwgRm72GQF

[37] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler, E. H. Chi, T. Hashimoto, O. Vinyals, P. Liang, J. Dean, and W. Fedus, "Emergent Abilities of Large Language Models," *Transactions on Machine Learning Research*, 2022, survey Certification. [Online]. Available: https://openreview.net/forum?id=yzkSU5zdwD

[38] R. Schaeffer, B. Miranda, and S. Koyejo, "Are Emergent Abilities of Large Language Models a Mirage?" in *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. [Online]. Available: https://openreview.net/forum?id=ITw9edRDlD

[39] J. Wei, M. Bosma, V. Zhao, K. Guu, A. W. Yu, B. Lester, N. Du, A. M. Dai, and Q. V. Le, "Finetuned Language Models are Zero-Shot Learners," in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=gEZrGCozdqR

[40] J. Wei, X. Wang, D. Schuurmans, M. Bosma, b. ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-Thought Prompting Elicits Reasoning in Large

Language Models," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 24 824–24 837. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/9d5609613524ecf4f15af0f7b31abca4-Paper-Conference.pdf

[41] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large Language Models are Zero-Shot Reasoners," in *Advances in Neural Information Processing Systems*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., vol. 35. Curran Associates, Inc., 2022, pp. 22 199–22 213. [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2022/file/8bb0d291acd4acf06ef112099c16f326-Paper-Conference.pdf

[42] Z. Ji, N. Lee, R. Frieske, T. Yu, D. Su, Y. Xu, E. Ishii, Y. J. Bang, A. Madotto, and P. Fung, "Survey of Hallucination in Natural Language Generation," *ACM Comput. Surv.*, vol. 55, no. 12, Mar. 2023. [Online]. Available: https://doi.org/10.1145/3571730

[43] J. Lin, R. Nogueira, and A. Yates, "Pretrained Transformers for Text Ranking: BERT and Beyond," 2021. [Online]. Available: https://arxiv.org/abs/2010.06467

[44] C. D. Manning, P. Raghavan, and H. Schütze, *Introduction to Information Retrieval*. USA: Cambridge University Press, 2008. [Online]. Available: https://nlp.stanford.edu/IR-book/information-retrieval-book.html

[45] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," *Found. Trends Inf. Retr.*, vol. 3, no. 4, p. 333–389, apr 2009. [Online]. Available: https://doi.org/10.1561/1500000019

[46] T. Mikolov, W.-t. Yih, and G. Zweig, "Linguistic Regularities in Continuous Space Word Representations," in *Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, L. Vanderwende, H. Daumé III, and K. Kirchhoff, Eds. Atlanta, Georgia: Association for Computational Linguistics, Jun. 2013, pp. 746–751. [Online]. Available: https://aclanthology.org/N13-1090

[47] T. Mikolov, K. Chen, G. Corrado, and J. Dean, "Efficient Estimation of Word Representations in Vector Space," 2013. [Online]. Available: https://arxiv.org/abs/1301.3781

[48] J. Pennington, R. Socher, and C. Manning, "GloVe: Global Vectors for Word Representation," in *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, A. Moschitti, B. Pang, and W. Daelemans, Eds. Doha, Qatar: Association for Computational Linguistics, Oct. 2014, pp. 1532–1543. [Online]. Available: https://aclanthology.org/D14-1162

[49] Q. Liu, M. J. Kusner, and P. Blunsom, "A Survey on Contextual Embeddings," 2020. [Online]. Available: https://arxiv.org/abs/2003.07278

[50] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, K. Inui, J. Jiang, V. Ng, and X. Wan, Eds. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 3982–3992. [Online]. Available: https://aclanthology.org/D19-1410

[51] V. Karpukhin, B. Oguz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense Passage Retrieval for Open-Domain Question Answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, B. Webber, T. Cohn, Y. He, and Y. Liu, Eds. Online: Association for Computational Linguistics, Nov. 2020, pp. 6769–6781. [Online]. Available: https://aclanthology.org/2020.emnlp-main.550

[52] O. Khattab and M. Zaharia, "ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT," in *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '20. New York, NY, USA: Association for Computing Machinery, 2020, p. 39–48. [Online]. Available: https://doi.org/10.1145/3397271.3401075

[53] J. Johnson, M. Douze, and H. Jégou, "Billion-Scale Similarity Search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021. [Online]. Available: https://doi.org/10.1109/TBDATA.2019.2921572

[54] J. J. Pan, J. Wang, and G. Li, "Survey of Vector Database Management Systems," *The VLDB Journal*, Jul 2024. [Online]. Available: https://doi.org/10.1007/s00778-024-00864-x

[55] Y. A. Malkov and D. A. Yashunin, "Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, p. 824–836, apr 2020. [Online]. Available: https://doi.org/10.1109/TPAMI.2018.2889473

[56] M. Aumüller, E. Bernhardsson, and A. Faithfull, "ANN-Benchmarks: A Benchmarking Tool for Approximate Nearest Neighbor Algorithms," *Information Systems*, vol. 87, p. 101374, 2020. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0306437918303685

[57] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval Augmentation Reduces Hallucination in Conversation," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Punta Cana, Dominican Republic: Association for

Computational Linguistics, Nov. 2021, pp. 3784–3803. [Online]. Available: https://aclanthology.org/2021.findings-emnlp.320

[58] O. Ovadia, M. Brief, M. Mishaeli, and O. Elisha, "Fine-Tuning or Retrieval? Comparing Knowledge Injection in LLMs," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 237–250. [Online]. Available: https://aclanthology.org/2024.emnlp-main.15/

[59] K. Guu, K. Lee, Z. Tung, P. Pasupat, and M.-W. Chang, "REALM: Retrieval-Augmented Language Model Pre-Training," in *Proceedings of the 37th International Conference on Machine Learning*, ser. ICML'20. JMLR.org, 2020. [Online]. Available: https://proceedings.mlr.press/v119/guu20a.html

[60] S. Borgeaud, A. Mensch, J. Hoffmann, T. Cai, E. Rutherford, K. Millican, G. B. Van Den Driessche, J.-B. Lespiau, B. Damoc, A. Clark, D. De Las Casas, A. Guy, J. Menick, R. Ring, T. Hennigan, S. Huang, L. Maggiore, C. Jones, A. Cassirer, A. Brock, M. Paganini, G. Irving, O. Vinyals, S. Osindero, K. Simonyan, J. Rae, E. Elsen, and L. Sifre, "Improving Language Models by Retrieving from Trillions of Tokens," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri, S. Jegelka, L. Song, C. Szepesvari, G. Niu, and S. Sabato, Eds., vol. 162. PMLR, 17–23 Jul 2022, pp. 2206–2240. [Online]. Available: https://proceedings.mlr.press/v162/borgeaud22a.html

[61] O. Ram, Y. Levine, I. Dalmedigos, D. Muhlgay, A. Shashua, K. Leyton-Brown, and Y. Shoham, "In-Context Retrieval-Augmented Language Models," *Transactions of the Association for Computational Linguistics*, vol. 11, pp. 1316–1331, 2023. [Online]. Available: https://aclanthology.org/2023.tacl-1.75

[62] W. Shi, S. Min, M. Yasunaga, M. Seo, R. James, M. Lewis, L. Zettlemoyer, and W.-t. Yih, "REPLUG: Retrieval-Augmented Black-Box Language Models," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 8364–8377. [Online]. Available: https://aclanthology.org/2024.naacl-long.463

[63] O. Khattab, K. Santhanam, X. L. Li, D. Hall, P. Liang, C. Potts, and M. Zaharia, "Demonstrate-Search-Predict: Composing retrieval and language models for knowledge-intensive NLP," 2023. [Online]. Available: https://arxiv.org/abs/2212.14024

[64] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the Middle: How Language Models Use Long Contexts,"

Transactions of the Association for Computational Linguistics, vol. 12, pp. 157–173, 02 2024. [Online]. Available: https://doi.org/10.1162/tacl_a_00638

[65] H. Yu, A. Gan, K. Zhang, S. Tong, Q. Liu, and Z. Liu, "Evaluation of Retrieval-Augmented Generation: A Survey," in *Big Data*, W. Zhu, H. Xiong, X. Cheng, L. Cui, Z. Dou, J. Dong, S. Pang, L. Wang, L. Kong, and Z. Chen, Eds. Singapore: Springer Nature Singapore, 2025, pp. 102–120. [Online]. Available: https://doi.org/10.1007/978-981-96-1024-2_8

[66] J. Chen, H. Lin, X. Han, and L. Sun, "Benchmarking Large Language Models in Retrieval-Augmented Generation," in *Proceedings of the Thirty-Eighth AAAI Conference on Artificial Intelligence and Thirty-Sixth Conference on Innovative Applications of Artificial Intelligence and Fourteenth Symposium on Educational Advances in Artificial Intelligence*, ser. AAAI'24/IAAI'24/EAAI'24. AAAI Press, 2024. [Online]. Available: https://doi.org/10.1609/aaai.v38i16.29728

[67] M. Karpinska, N. Akoury, and M. Iyyer, "The Perils of Using Mechanical Turk to Evaluate Open-Ended Text Generation," in *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Online and Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 1265–1285. [Online]. Available: https://aclanthology.org/2021.emnlp-main.97/

[68] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: a Method for Automatic Evaluation of Machine Translation," in *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics*, ser. ACL '02. USA: Association for Computational Linguistics, 2002, p. 311–318. [Online]. Available: https://doi.org/10.3115/1073083.1073135

[69] C.-Y. Lin, "ROUGE: A Package for Automatic Evaluation of Summaries," in *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, Jul. 2004, pp. 74–81. [Online]. Available: https://aclanthology.org/W04-1013/

[70] T. Zhang*, V. Kishore*, F. Wu*, K. Q. Weinberger, and Y. Artzi, "BERTScore: Evaluating Text Generation with BERT," in *International Conference on Learning Representations*, 2020. [Online]. Available: https://openreview.net/forum?id=SkeHuCVFDr

[71] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo, "A Survey on LLM-as-a-Judge," 2025. [Online]. Available: https://arxiv.org/abs/2411.15594

[72] C.-H. Chiang and H.-y. Lee, "Can Large Language Models Be an Alternative to Human Evaluations?" in *Proceedings of the 61st Annual Meeting of the*

Association for Computational Linguistics (Volume 1: Long Papers), A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 15 607–15 631. [Online]. Available: https://aclanthology.org/2023.acl-long.870/

[73] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 338–354. [Online]. Available: https://aclanthology.org/2024.naacl-long.20

[74] H. Chase, "LangChain," Oct. 2022, Accessed: Oct. 18, 2025. [Online]. Available: https://github.com/langchain-ai/langchain

[75] LangChain, "LangSmith," Accessed: Oct. 18, 2025. [Online]. Available: https://www.langchain.com/langsmith

[76] E. Dilocker, B. van Luijt, B. Voorbach, M. S. Hasan, A. Rodriguez, D. A. Kulawiak, M. Antas, and P. Duckworth, "Weaviate," Accessed: Oct. 18, 2025. [Online]. Available: https://github.com/weaviate/weaviate

[77] Ollama Contributors, "Ollama," Accessed: Oct. 18, 2025. [Online]. Available: https://github.com/ollama/ollama

[78] OpenAI, "Hello GPT-4o," 2024, Accessed: Oct. 18, 2025. [Online]. Available: https://openai.com/index/hello-gpt-4o/

[79] ——, "New embedding models and API updates," 2024, Accessed: Sep. 11, 2025. [Online]. Available: https://openai.com/index/new-embedding-models-and-api-updates/

[80] LangChain, "Build a Retrieval Augmented Generation (RAG) App," 2024, Accessed: Oct. 18, 2025. [Online]. Available: https://python.langchain.com/v0.2/docs/tutorials/rag/

[81] P. Ginsparg, "arXiv," Accessed: Aug. 4, 2025. [Online]. Available: https://arxiv.org/

[82] E. Saravia, "ML Papers of the Week," Accessed: Aug. 4, 2025. [Online]. Available: https://github.com/dair-ai/ML-Papers-of-the-Week

[83] L. Schwab, "arxiv.py," Accessed: Aug. 4, 2025. [Online]. Available: https://github.com/lukasschwab/arxiv.py

[84] M. Fenniak, M. Stamy, pubpub zz, M. Thoma, M. Peveler, exiledkingcc, and pypdf Contributors, "The pypdf Library," 2024, see https://pypdf.readthedocs.io/en/latest/meta/CONTRIBUTORS.html

for all contributors, Accessed: Sep. 13, 2025. [Online]. Available: https://pypi.org/project/pypdf/

[85] M. Khalusova, "Chunking for RAG: Best Practices," 2024, Accessed: Sep. 13, 2025. [Online]. Available: https://unstructured.io/blog/chunking-for-rag-best-practices

[86] OpenAI, "New and Improved Embedding Model," 2022, Accessed: Sep. 11, 2025. [Online]. Available: https://openai.com/index/new-and-improved-embedding-model/

[87] Unstructured Contributors, "Unstructured," Accessed: Sep. 13, 2025. [Online]. Available: https://github.com/Unstructured-IO/unstructured

[88] B. Smith and A. Troynikov, "Evaluating Chunking Strategies for Retrieval," Chroma, Tech. Rep., July 2024, accessed: Sep. 13, 2025. [Online]. Available: https://research.trychroma.com/evaluating-chunking

[89] OpenAI, "What are tokens and how to count them?" Accessed: Sep. 13, 2025. [Online]. Available: https://help.openai.com/en/articles/4936856-what-are-tokens-and-how-to-count-them

[90] A. J. Yepes, Y. You, J. Milczek, S. Laverde, and R. Li, "Financial Report Chunking for Effective Retrieval Augmented Generation," 2024. [Online]. Available: https://arxiv.org/abs/2402.05131

[91] S. Xiao, Z. Liu, P. Zhang, N. Muennighoff, D. Lian, and J.-Y. Nie, "C-Pack: Packed Resources For General Chinese Embeddings," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 641–649. [Online]. Available: https://doi.org/10.1145/3626772.3657878

[92] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang, "Towards General Text Embeddings with Multi-stage Contrastive Learning," 2023. [Online]. Available: https://arxiv.org/abs/2308.03281

[93] X. Zhang, Y. Zhang, D. Long, W. Xie, Z. Dai, J. Tang, H. Lin, B. Yang, P. Xie, F. Huang, M. Zhang, W. Li, and M. Zhang, "mGTE: Generalized Long-Context Text Representation and Reranking Models for Multilingual Text Retrieval," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, F. Dernoncourt, D. Preoţiuc-Pietro, and A. Shimorina, Eds. Miami, Florida, US: Association for Computational Linguistics, Nov. 2024, pp. 1393–1412. [Online]. Available: https://aclanthology.org/2024.emnlp-industry.103/

[94] E. Cardenas, "Hybrid Search Explained," 2025, Accessed: Jul. 19, 2025. [Online]. Available: https://weaviate.io/blog/hybrid-search-explained

[95] X. Wang, Z. Wang, X. Gao, F. Zhang, Y. Wu, Z. Xu, T. Shi, Z. Wang, S. Li, Q. Qian, R. Yin, C. Lv, X. Zheng, and X. Huang, "Searching for Best Practices in Retrieval-Augmented Generation," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 17 716–17 736. [Online]. Available: https://aclanthology.org/2024.emnlp-main.981

[96] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised Cross-lingual Representation Learning at Scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 8440–8451. [Online]. Available: https://aclanthology.org/2020.acl-main.747/

[97] K. Santhanam, O. Khattab, J. Saad-Falcon, C. Potts, and M. Zaharia, "ColBERTv2: Effective and Efficient Retrieval via Lightweight Late Interaction," in *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, M. Carpuat, M.-C. de Marneffe, and I. V. Meza Ruiz, Eds. Seattle, United States: Association for Computational Linguistics, Jul. 2022, pp. 3715–3734. [Online]. Available: https://aclanthology.org/2022.naacl-main.272/

[98] B. Clavié, "RAGatouille," Accessed: Jul. 23, 2025. [Online]. Available: https://github.com/AnswerDotAI/RAGatouille

[99] X. Ma, X. Zhang, R. Pradeep, and J. Lin, "Zero-Shot Listwise Document Reranking with a Large Language Model," 2023. [Online]. Available: https://arxiv.org/abs/2305.02156

[100] Q. Leng, K. Uhlenhuth, and A. Polyzotis, "Best Practices for LLM Evaluation of RAG Applications," 2023, Accessed: Aug. 12, 2025. [Online]. Available: https://www.databricks.com/blog/LLM-auto-eval-best-practices-RAG

[101] S. Kulkarni and A. Savelieva, "The path to a golden dataset, or how to evaluate your RAG?" 2024, Accessed: Aug. 12, 2025. [Online]. Available: https://medium.com/data-science-at-microsoft/the-path-to-a-golden-dataset-or-how-to-evaluate-your-rag-045e23d1f13f

[102] H. Selbie and T. Pakeman, "Optimizing RAG retrieval: Test, tune, succeed," December 2024, Accessed: Aug. 12, 2025. [Online]. Available: https://cloud.google.com/blog/products/ai-machine-learning/optimizing-rag-retrieval

[103] S. Li, L. Stenzel, C. Eickhoff, and S. A. Bahrainian, "Enhancing Retrieval-Augmented Generation: A Study of Best Practices," in

Proceedings of the 31st International Conference on Computational Linguistics, O. Rambow, L. Wanner, M. Apidianaki, H. Al-Khalifa, B. D. Eugenio, and S. Schockaert, Eds. Abu Dhabi, UAE: Association for Computational Linguistics, Jan. 2025, pp. 6705–6717. [Online]. Available: https://aclanthology.org/2025.coling-main.449/

[104] L. Gao, X. Ma, J. Lin, and J. Callan, "Precise Zero-Shot Dense Retrieval without Relevance Labels," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, A. Rogers, J. Boyd-Graber, and N. Okazaki, Eds. Toronto, Canada: Association for Computational Linguistics, Jul. 2023, pp. 1762–1777. [Online]. Available: https://aclanthology.org/2023.acl-long.99

[105] N. S. Adhikari and S. Agarwal, "A Comparative Study of PDF Parsing Tools Across Diverse Document Categories," 2025. [Online]. Available: https://arxiv.org/abs/2410.09871

[106] OpenAI, "Introducing GPT-4.1 in the API," 2025, Accessed: Sep. 11, 2025. [Online]. Available: https://openai.com/index/gpt-4-1/

[107] S. Roychowdhury, S. Soman, H. G. Ranjani, N. Gunda, V. Chhabra, and S. K. BALA, "Evaluation of RAG Metrics for Question Answering in the Telecom Domain," in *ICML 2024 Workshop on Foundation Models in the Wild*, 2024. [Online]. Available: https://openreview.net/forum?id=L74piNoToX

[108] S. Barnett, S. Kurniawan, S. Thudumu, Z. Brannelly, and M. Abdelrazek, "Seven Failure Points When Engineering a Retrieval Augmented Generation System," in *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering - Software Engineering for AI*, ser. CAIN '24. New York, NY, USA: Association for Computing Machinery, 2024, p. 194–199. [Online]. Available: https://doi.org/10.1145/3644815.3644945

[109] GROBID Contributors, "GROBID," https://github.com/kermitt2/grobid, 2008–2025, Accessed: Sep. 24, 2025.

[110] L. Blecher, G. Cucurull, T. Scialom, and R. Stojnic, "Nougat: Neural Optical Understanding for Academic Documents," in *The Twelfth International Conference on Learning Representations*, 2024. [Online]. Available: https://openreview.net/forum?id=fUtxNAKpdV

[111] OmniAI, "OmniAI OCR Benchmark," 2025, Accessed: Sep. 24, 2025. [Online]. Available: https://getomni.ai/blog/ocr-benchmark

[112] M. Faysse, H. Sibille, T. Wu, B. Omrani, G. Viaud, C. HUDELOT, and P. Colombo, "ColPali: Efficient Document Retrieval with Vision Language Models," in *The Thirteenth International Conference on Learning Representations*, 2025. [Online]. Available: https://openreview.net/forum?id=ogjBpZ8uSi

[113] J. Lála, O. O'Donoghue, A. Shtedritski, S. Cox, S. G. Rodriques, and A. D. White, "PaperQA: Retrieval-Augmented Generative Agent for Scientific Research," 2023. [Online]. Available: https://arxiv.org/abs/2312.07559

[114] M. D. Skarlinski, S. Cox, J. M. Laurent, J. D. Braza, M. Hinks, M. J. Hammerling, M. Ponnapati, S. G. Rodriques, and A. D. White, "Language agents achieve superhuman synthesis of scientific knowledge," 2024. [Online]. Available: https://arxiv.org/abs/2409.13740

[115] A. Asai, J. He, R. Shao, W. Shi, A. Singh, J. C. Chang, K. Lo, L. Soldaini, S. Feldman, M. D'arcy, D. Wadden, M. Latzke, M. Tian, P. Ji, S. Liu, H. Tong, B. Wu, Y. Xiong, L. Zettlemoyer, G. Neubig, D. Weld, D. Downey, W. tau Yih, P. W. Koh, and H. Hajishirzi, "OpenScholar: Synthesizing Scientific Literature with Retrieval-augmented LMs," 2024. [Online]. Available: https://arxiv.org/abs/2411.14199

[116] Y. Zheng, S. Sun, L. Qiu, D. Ru, C. Jiayang, X. Li, J. Lin, B. Wang, Y. Luo, R. Pan, Y. Xu, Q. Min, Z. Zhang, Y. Wang, W. Li, and P. Liu, "OpenResearcher: Unleashing AI for Accelerated Scientific Research," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*, D. I. Hernandez Farias, T. Hope, and M. Li, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 209–218. [Online]. Available: https://aclanthology.org/2024.emnlp-demo.22/